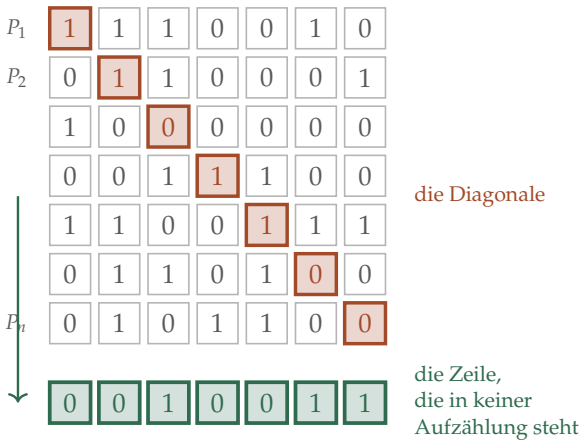
INFORMATIK VON GRUND AUF — BEGLEITBAND

Der seltsame Kreis

Selbstbezug, Universalität und Grenze



Der rote Faden durch sechzehn Bände

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Informatik von Grund auf — Begleitband
Der seltsame Kreis · Selbstbezug, Universalität und Grenze

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Begleitband: Der seltsame Kreis*. Fassung 1.0.1.
`https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 124 Seiten.

Stand der Forschung: August 2026

Inhaltsverzeichnis

Vorwort 1

I Die Bewegung 9

1 Die Diagonale 11

2 Der Selbstbezug 17

3 Universalität 25

II Die Grenzen 33

4 Gödel 35

5 Turing 41

6 Wo die Diagonale versagt 47

III Der Kreis in der Technik 55

7 Der Übersetzer, der sich selbst übersetzt 57

8 Das Programm, das sich selbst kopiert 65

9 Das System, das sich selbst beschreibt 73

IV Der Kreis im Denken 81

10 Das Gehirn, das sich selbst modelliert 83

11 Die Grenze anderer Bauart 91

12 Was die Reihe gezeigt hat 97

V Anhang **105**

A Die Bewegung im Überblick **107**

B Landkarte **109**

C Glossar **113**

D Zeittafel **115**

E Kommentierte Quellen **117**

Vorwort: Eine Bewegung, sechzehn Bände

Dieser Band bringt nichts Neues. *Er zeigt, dass in den sechzehn vorangegangenen Bänden immer wieder dieselbe Bewegung vorkam — und dass sie zugleich die Stärke und die Grenze des Rechnens erklärt.*

Die Behauptung

Universalität	Ein System ist mächtig genug, jedes andere zu beschreiben — also auch sich selbst
Selbstbezug	Damit lässt sich eine Aussage bilden, die von sich selbst spricht
Grenze	**Diese Aussage lässt sich so wählen, dass das System an ihr scheitert**

Die drei Zeilen hängen zusammen, und zwar in dieser Reihenfolge. Die Grenze ist keine Schwäche, die man durch ein besseres System beheben könnte. Sie folgt aus der Mächtigkeit selbst — wer sie loswerden will, muss auf die Universalität verzichten.

Deshalb der Titel. Das Bild vom seltsamen Kreis stammt von Douglas Hofstadter*: eine Hierarchie, die man aufwärts durchläuft und bei der man wieder unten ankommt. Genau das geschieht, wenn ein Beschreibungssystem sich selbst beschreibt.*

Wo die Bewegung in dieser Reihe vorkam.

Band	Gegenstand	Die Grenze
0	Aussagenlogik	— (entscheidbar)
1	Prädikatenlogik	**unentscheidbar**
2	Berechenbarkeit	**Halteproblem**
3	Komplexität	Hierarchiesätze
4	Architektur	universelle Maschine
6	Übersetzerbau	**Bootstrapping**
7	Information	Kolmogorow-Komplexität
10	Datenbanken	**Katalog im Katalog**
12	Symbolische KI	Selbstanwendung
13	Lernen	Modell des Modells
14	Gehirn	**Selbstmodellierung**
15	Quanteninformatik	**Auslesegrenze**

Die letzte Zeile fällt aus der Reihe, und darauf kommt Kapitel 11 zurück: Die Grenze der Quantenmechanik entsteht nicht durch Selbstbezug, sondern durch Komplementarität. Es ist eine Grenze anderer Bauart, und der Vergleich schärft beide.

Was dieser Band nicht behauptet

Über Gödels Sätze wird mehr Unsinn geschrieben als über fast jedes andere mathematische Ergebnis.

Behauptung	Urteil	
Es gibt Wahrheiten, die niemand beweisen kann	irreführend	relativ zum System
Der Mensch übertrifft die Maschine	unbelegt	Kapitel 10
Alles ist relativ	falsch	
Kein System kann sich selbst verstehen	falsch	**sehr wohl, nur nicht vollständig**
Mathematik ist widersprüchlich	falsch	

> *Die vierte Zeile ist die wichtigste Berichtigung. Ein hinreichend mächtiges System kann sehr wohl über sich selbst sprechen — genau das ist die Voraussetzung des Beweises. Was es nicht kann, ist* vollständig *und* widerspruchsfrei *zugleich über sich sprechen.*

Der Unterschied ist nicht spitzfindig. Die falsche Fassung klingt nach einem Geheimnis; die richtige ist eine präzise Aussage über formale Systeme, und sie hat handfeste Folgen für Übersetzer, Datenbanken und Programmprüfung.

Was dieses Buch will

Erstens: Die Bewegung selbst zeigen.
Kapitel 1 rechnet das Diagonalverfahren an einer Tabelle mit sieben Zeilen vollständig durch. Wer es einmal von Hand gemacht hat, erkennt es danach überall wieder — bei Cantor, bei Gödel, bei Turing und in den Hierarchiesätzen.

Zweitens: Die Grenzen ehrlich benennen.

> *Kapitel 6 zeigt, wo das Verfahren* versagt *— nämlich bei P gegen NP. Das ist lehrreicher als alle Erfolge zusammen, denn es sagt, warum die wichtigste offene Frage der Informatik offen ist.*

Drittens: Den Kreis in der Praxis zeigen.
Ein Übersetzer, der sich selbst übersetzt. Ein Programm, das sich selbst ausgibt. Ein Datenbankkatalog, der sich selbst enthält. Diese Dinge sind keine Kuriositäten, sondern die alltägliche Form derselben Bewegung.

Drei Warnungen

Erstens: Unentscheidbar heißt nicht unlösbar.
Das Halteproblem ist unentscheidbar, und trotzdem prüfen Werkzeuge täglich Millionen Programme. Sie geben nur manchmal keine Antwort — und das genügt.
Zweitens: Die Grenzen sind Aussagen über den schlimmsten Fall.

> *Ein Ergebnis der Form „es gibt kein Verfahren, das alle Fälle löst" sagt nichts darüber, wie viele Fälle ein gutes Verfahren löst. In der Praxis sind es fast alle, die vorkommen.*

Drittens: Aus Gödel folgt nichts über Bewusstsein.
Die entsprechenden Argumente sind seit sechzig Jahren im Umlauf und ebenso lange widerlegt. Kapitel 10 führt beides vor.

Zum Aufbau

- **Teil I** (Kap. 1–3): Die Bewegung. Die Diagonale, der Selbstbezug, die Universalität.
- **Teil II** (Kap. 4–6): Die Grenzen. Gödel, Turing, die Hierarchien — und wo das Verfahren versagt.
- **Teil III** (Kap. 7–9): Der Kreis in der Technik. Übersetzer, Selbstkopie, Selbstbeschreibung.
- **Teil IV** (Kap. 10–12): Der Kreis im Denken. Das Gehirn, die Messung, die Bilanz.

Ein Brief von 1902

GOTTLOB FREGE hatte sein Lebenswerk fast vollendet: eine Begründung der Arithmetik allein aus der Logik. Der zweite Band war im Druck.

Der Brief	BERTRAND RUSSELL schrieb ihm im Juni 1902
Der Inhalt	Eine Frage über die Menge aller Mengen, die sich nicht selbst enthalten
Die Folge	Freges System war widersprüchlich

Die Frage. Sei $R = \{x : x \notin x\}$. Gilt $R \in R$?

Annahme	Dann	Also
$R \in R$	erfüllt R die Bedingung nicht	$R \notin R$
$R \notin R$	erfüllt R die Bedingung	$R \in R$

Beide Annahmen führen auf ihr Gegenteil. Zwei Zeilen genügten, um ein Werk von zwei Jahrzehnten zu erschüttern — und Frege druckte einen Nachtrag, in dem er das offen einräumte. Das ist eine der ehrenhaftesten Reaktionen der Wissenschaftsgeschichte.

Warum das hierher gehört.

Das Muster	Ein Begriff wird auf sich selbst angewandt und kehrt sich um
Bei Cantor	Eine Aufzählung wird auf ihre eigene Diagonale angewandt
Bei Gödel	Eine Beweisbarkeitsaussage spricht über sich selbst
Bei Turing	Ein Prüfprogramm wird auf sich selbst angewandt

Viermal derselbe Griff. Russells Antinomie ist die einfachste Fassung, sie kommt ohne Zahlen und ohne Maschinen aus — und deshalb steht sie am Anfang dieses Bandes.

Neunzig Jahre Selbstbezug

≈600 v. Chr.	Epimenides	der lügende Kreter
1891	**Cantor**	**das Diagonalverfahren**
1902	Russell	der Brief an Frege
1910–13	Whitehead, Russell	Typentheorie als Ausweg
1928	Hilbert	das Entscheidungsproblem
1931	**Gödel**	**Unvollständigkeit**
1935	Kleene	der Rekursionssatz
1936	**Turing**	**das Halteproblem**
1936	Church	das Lambda-Kalkül
1944	Post	Grade der Unlösbarkeit
1946	von Neumann	selbstreproduzierende Automaten
1953	**Rice**	**alle nichttrivialen Eigenschaften**
1958	Nagel, Newman	Gödel für Nichtfachleute
1961	Lucas	Gödel gegen die Maschine
1965	**Hartmanis, Stearns**	**Zeithierarchie**
1971	Cook	NP-Vollständigkeit
1974	Chaitin	Unvollständigkeit aus Informationsmangel
1975	**Baker, Gill, Solovay**	**Diagonalisierung genügt nicht**
1976	Dawkins	der Selbstkopierer als Bild
1979	**Hofstadter**	**Gödel, Escher, Bach**
1984	**Thompson**	**Reflections on Trusting Trust**
1989	Penrose	erneut Gödel gegen die Maschine
2005	Franzén	die Aufräumarbeit
2007	Hofstadter	I Am a Strange Loop

Zwei Zeilen verdienen eine Anmerkung.
1975, Baker, Gill und Solovay. Sie zeigten, dass das Diagonalverfahren die Frage P gegen NP nicht entscheiden kann — nicht, weil man es noch nicht geschickt genug angestellt hätte, sondern grundsätzlich. Ein Beweis darüber, welche Beweise unmöglich sind. Kapitel 6 führt ihn vor.

1984, Thompson. Ein Übersetzer, der eine Hintertür in jedes Programm einbaut und die Hintertür auch in jeden neu übersetzten Übersetzer schreibt — sodass sie im Quelltext nirgends steht. Der Selbstbezug wird hier zum Sicherheitsproblem, und Kapitel 7 zeigt, dass es keine vollständige Abhilfe gibt.

Ein System, das mächtig genug ist, über sich selbst zu sprechen,
ist mächtig genug, an sich selbst zu scheitern.
Beides folgt aus derselben Eigenschaft,
und man kann nicht das eine ohne das andere haben.

— die Behauptung dieses Bandes in vier Zeilen.
Die folgenden zwölf Kapitel prüfen sie an
Logik, Berechenbarkeit, Komplexität, Übersetzern,
Datenbanken, Gehirnen und Qubits.

Part I

Die Bewegung

1 Die Diagonale

Ein Verfahren in drei Schritten

1	Nimm an, es gäbe eine Aufzählung aller Objekte einer Art.
2	Baue ein neues Objekt, das sich vom n-ten an der Stelle n unterscheidet.
3	Es kann in keiner Zeile stehen — Widerspruch.

Drei Zeilen, und sie tragen die halbe theoretische Informatik. Der ganze Trick steckt in Schritt 2: Man muss aus der Aufzählung selbst ein Objekt bauen können, das von derselben Art ist. Wo das gelingt, gibt es eine Grenze; wo nicht, gibt es keine.

Cantors Anwendung, 1891.

Die Objekte	Unendliche Folgen aus 0 und 1
Die Annahme	Man könne sie durchnummerieren
Das neue Objekt	An Stelle n das Gegenteil der n-ten Folge
Der Schluss	**Es gibt überabzählbar viele**

Das ist die Titelgrafik dieses Bandes. Sieben Zeilen, die Diagonale markiert, darunter die umgekehrte Zeile — und sie steht mit Sicherheit in keiner der sieben, denn sie unterscheidet sich von jeder an mindestens einer Stelle.

Warum es nicht immer funktioniert

Man kann nicht mit demselben Griff zeigen, dass es überabzählbar viele natürliche Zahlen gibt.

Der Versuch	Zähle alle natürlichen Zahlen auf, drehe die Diagonalziffern um
Das Ergebnis	Eine *unendlich* lange Ziffernfolge
Der Fehler	**Das ist keine natürliche Zahl**

Genau hier scheitert das Verfahren, und daran erkennt man seine Bedingung: Das neue Objekt muss zur selben Art gehören wie die aufgezählten. Bei endlichen Zeichenketten geht das nicht, bei unendlichen Folgen schon.

Diese Bedingung ist der wunde Punkt bei jeder Anwendung — und Kapitel 6 zeigt, dass sie bei P gegen NP nicht erfüllbar ist.

Dieselbe Bewegung, vier Gestalten

	Aufgezählt	**Das neue Objekt**	**Folge**
Cantor	alle Folgen	die umgekehrte Diagonale	**über-abzählbar**
Russell	alle Mengen	$\{x : x \notin x\}$	Widerspruch
Gödel	alle Beweise	„ich bin unbeweisbar“	**unvoll-ständig**
Turing	alle Programme	„tu das Gegenteil“	**unent-scheidbar**

Vier der berühmtesten Ergebnisse der Mathematik, und sie sind derselbe Beweis in vier Anzügen. Wer einen versteht, versteht die anderen — vorausgesetzt, er erkennt, welche Rolle die Aufzählung jeweils spielt.

Was die Aufzählung jeweils leistet.

Cantor	wird als möglich *angenommen* und widerlegt
Russell	**ist gar keine Aufzählung** — die Selbstanwendung genügt
Gödel	**existiert wirklich**: Beweise sind aufzählbar
Turing	**existiert wirklich**: Programme sind aufzählbar

Der Unterschied zwischen den ersten beiden und den letzten beiden ist entscheidend. Bei Cantor endet das Argument mit einem Widerspruch zur Annahme. Bei Gödel und Turing gibt es die Aufzählung tatsächlich — der Widerspruch trifft also nicht sie, sondern das vorausgesetzte Entscheidungsverfahren.

Aufgabe 1.1.

(a) Führen Sie das Diagonalverfahren an einer Tabelle aus.
(b) Zeigen Sie, dass die reellen Zahlen überabzählbar sind.
(c) Untersuchen Sie, welche Mengen aufzählbar bleiben.

Lösung Schritt für Schritt.

(a) Das Verfahren an einer Tabelle.
Die Annahme. Diese sieben Folgen seien alle.

	1	2	3	4	5	6	7
f_1	**1**	1	1	0	0	1	0
f_2	0	**1**	1	0	0	0	1
f_3	1	0	**0**	0	0	0	0
f_4	0	0	1	**1**	1	0	0
f_5	1	1	0	0	**1**	1	1
f_6	0	1	1	0	1	**0**	0
f_7	0	1	0	1	1	0	**0**

Die Diagonale ablesen.

$$d = (1, 1, 0, 1, 1, 0, 0).$$

Umkehren.

$$g = (0, 0, 1, 0, 0, 1, 1).$$

Die Probe.

Zeile	$f_n(n)$	$g(n)$	verschieden?
f_1	1	0	✓
f_2	1	0	✓
f_3	0	1	✓
f_4	1	0	✓
f_5	1	0	✓
f_6	0	1	✓
f_7	0	1	✓

g unterscheidet sich von jeder Zeile an genau der Stelle, die ihre Nummer trägt. Also ist g keine der sieben — und da die Konstruktion für jede Aufzählung funktioniert, kann keine Aufzählung vollständig sein.

Bemerkenswert an der Rechnung ist ihre Sparsamkeit. Man prüft nicht sieben Zeilen auf Gleichheit mit g — das wären neunundvierzig Vergleiche. Man prüft je Zeile eine *Stelle, nämlich die diagonale, und das genügt.*

(b) Die reellen Zahlen.
Die Annahme. Die Zahlen in $[0, 1)$ seien aufzählbar.

	Dezimalbruch	Diagonalziffer	neue Ziffer
x_1	0,**3**14159 …	3	4
x_2	0,5**7**7215 …	7	8
x_3	0,69**3**147 …	3	4
x_4	0,414**2**13 …	2	3
x_5	0,61803**3** …	3	4

$$y = 0{,}48434\ldots$$

Prüfung	Stelle	
$y \neq x_1$	1	$4 \neq 3$
$y \neq x_2$	2	$8 \neq 7$
$y \neq x_3$	3	$4 \neq 3$

Eine Feinheit. Man muss die Ziffern 0 und 9 vermeiden.

> *Der Grund:* $0{,}4999\ldots = 0{,}5000\ldots$ *Zwei verschiedene Ziffernfolgen können dieselbe Zahl bezeichnen, und dann wäre y vielleicht doch eine der aufgezählten. Wählt man die neuen Ziffern nur aus* $\{2, \ldots, 7\}$*, so kann das nicht eintreten.*

Diese Feinheit wird in Darstellungen oft übergangen und ist der einzige Punkt, an dem der Beweis Sorgfalt verlangt. Sie ist auch lehrreich: Das Verfahren arbeitet mit Darstellungen, *nicht mit den Objekten selbst — und wo eine Darstellung mehrdeutig ist, muss man nachbessern.*

(c) Welche Mengen aufzählbar bleiben.

Menge	Mächtigkeit	
Natürliche Zahlen	abzählbar	
Ganze Zahlen	abzählbar	abwechselnd
Rationale Zahlen	abzählbar	**Cantors erstes Verfahren**
Endliche Zeichenketten	abzählbar	nach Länge ordnen
Alle Programme	**abzählbar**	**sind Zeichenketten**
Unendliche Folgen	überabzählbar	
Reelle Zahlen	überabzählbar	
Alle Funktionen $\mathbb{N} \to \{0,1\}$	**überabzählbar**	

> *Die fünfte und die letzte Zeile ergeben zusammen ein Ergebnis, das man sofort ablesen kann: Es gibt abzählbar viele Programme und überabzählbar viele Funktionen. Also sind fast alle Funktionen* nicht *berechenbar — und zwar ohne dass man ein einziges Beispiel angeben müsste.*

Die Abzählung.

Länge	Ketten	kumuliert	
1	128	128	
2	16 384	16 512	
3	$2{,}1 \cdot 10^6$	$2{,}1 \cdot 10^6$	
n	128^n	endlich	durchnummerierbar

Man ordnet erst nach Länge, dann alphabetisch. Jede einzelne Stufe ist endlich, also erhält jedes Programm eine endliche Nummer. Diese Abzählung ist die stillschweigende Voraussetzung von Gödel und Turing gleichermaßen.

Nichtkonstruktiv heißt nicht wertlos

Was gezeigt ist	Fast alle Funktionen sind unberechenbar
Was nicht gezeigt ist	Welche
Der Wert	Man weiß, dass man suchen muss — und wo nicht

Die Abzählung liefert kein einziges Beispiel für eine unberechenbare Funktion. Das erste konkrete Beispiel lieferte Turing, und es kostete ihn einen eigenen Kunstgriff — dieselbe Diagonale, aber angewandt auf Maschinen statt auf Folgen. Kapitel 5 führt ihn aus.

Cantor[3] führte das Verfahren 1891 ein; Rogers[22] stellt seine Anwendungen in der Rekursionstheorie systematisch dar; Sipser[24] bringt die für die Informatik wichtigste Fassung.

2 Der Selbstbezug

Wie ein Satz über sich selbst spricht

Die Schwierigkeit ist nicht, dass ein Satz auf sich zeigt. Sie ist, dass er es *ohne zu zeigen* tut.

Naiv	„Dieser Satz ist falsch“ — braucht das Wort „dieser“
Das Problem	In einem formalen System gibt es kein „dieser“
Der Ausweg	**Ein Satz, der seine eigene Beschreibung erzeugt**

Genau darin liegt Gödels technische Leistung, und sie wird in populären Darstellungen meist übersprungen. Der Satz „ich bin unbeweisbar“ ist leicht hingeschrieben; ihn in der Sprache der Arithmetik zu bilden, ohne auf ihn zu zeigen, ist die eigentliche Arbeit.

Quines Kunstgriff.

Der Satz	„ergibt eine Falschheit, wenn man es seinem eigenen Zitat voranstellt“ **ergibt eine Falschheit, wenn man es seinem eigenen Zitat voranstellt**

Der Bau	Ein Ausdruck in Anführungszeichen, gefolgt von demselben Ausdruck ohne
Die Wirkung	Der Satz spricht über sich, ohne das Wort „dieser“ zu benutzen
Der Preis	**Er ist doppelt so lang wie sein Gegenstand**

Die dritte Zeile ist die technische Pointe. Ein Objekt, das seine eigene Beschreibung enthält, muss größer sein als sie — es sei denn, es erzeugt sie, statt sie zu speichern. Genau das tun Quines, der Rekursionssatz und Gödels Konstruktion.

Der Rekursionssatz

Definition 2.1. Zu jedem berechenbaren f gibt es ein Programm p mit

$$\text{Programm } p \text{ verhält sich wie } f(p).$$

Kurz: **Jedes Programm kann seinen eigenen Quelltext benutzen.**

Was das erlaubt	Ein Programm darf „sei *s* mein eigener Quelltext“ schreiben
Wie es geht	Es liest sich nicht aus einer Datei — **es erzeugt sich**
Wozu es dient	Selbstkopierer, Quines, Turings Beweis, Viren

Der Satz klingt harmlos und ist ein Werkzeug von großer Reichweite. Er sagt: Selbstbezug ist in jedem universellen Berechnungsmodell verfügbar, umsonst und ohne Zusatzannahmen. Man muss ihn nicht einbauen — er ist schon da.

Und daraus folgt die Kernbehauptung dieses Bandes unmittelbar: Wer Universalität hat, hat Selbstbezug. Wer Selbstbezug hat, hat die Diagonale. Wer die Diagonale hat, hat die Grenze.

Nicht jeder Selbstbezug ist gefährlich

Beispiel	Problem?	
„Dieser Satz hat fünf Wörter“	nein	schlicht wahr
„Dieser Satz ist wahr“	harmlos	beides möglich
„Dieser Satz ist falsch“	**Widerspruch**	
Rekursive Funktion	nein	Band 1
Quine	nein	gibt es wirklich

Selbstbezug allein erzeugt keinen Widerspruch. Es braucht zusätzlich eine Umkehrung *— ein „nicht“, das den Bezug gegen sich selbst wendet. Ohne diese Umkehrung ist Selbstbezug ein alltägliches und nützliches Mittel.*

Deshalb ist Gödels Satz nicht „ich bin falsch“, sondern „ich bin unbeweisbar“. Der Unterschied rettet die Widerspruchsfreiheit: Der Satz ist wahr und unbeweisbar, statt weder wahr noch falsch.

Aufgabe 2.1.

(a) Schreiben Sie ein Programm, das sich selbst ausgibt.
(b) Prüfen Sie den Rekursionssatz an einem Beispiel.
(c) Untersuchen Sie, wann Selbstbezug widersprüchlich wird.

Lösung Schritt für Schritt.

(a) Ein Quine.
Der falsche Ansatz. Die Datei lesen:

```
print(open(__file__).read())
```

Das ist geschummelt. Das Programm gibt sich nicht selbst aus, sondern eine Datei, die zufällig es enthält. Auf einem System ohne Dateien versagt es.

Der Bau eines echten Quine.

1	Der Text besteht aus zwei Teilen: einer Zeichenkette und Code.
2	Die Zeichenkette enthält den Code als Text.
3	Der Code gibt die Zeichenkette **zweimal** aus: einmal in Anführungszeichen, einmal ohne.

In Python.

```
s = 's = %r\nprint(s %% s)'
print(s % s)
```

Die Ausführung Schritt für Schritt.

Zeile 1	*s* erhält den Text „s = %r, Zeilenumbruch, print(s % s)“
Zeile 2	`%r` ersetzt die Stelle durch *s* **in Anführungszeichen**
Ergebnis	Genau die beiden Zeilen des Programms

Der Kern ist das `%r`: Es fügt die Zeichenkette mit *ihren Anführungszeichen ein. Damit erzeugt Zeile 1 sich selbst, und Zeile 2 erzeugt sich ebenfalls, weil sie im Text von s steht. Das ist Quines Kunstgriff in vier Zeichen.*

Die Längenbilanz.

Sprache	Zeichen	Anteil Code	
Python	30	50 %	
C	≈ 100	50 %	
Lambda-Kalkül	≈ 20	—	sehr kurz

In jeder universellen Sprache gibt es Quines — das garantiert der Rekursionssatz. Ihre Länge hängt davon ab, wie bequem sich Zeichenketten zitieren lassen.

(b) Der Rekursionssatz an einem Beispiel.
Die Aufgabe. Ein Programm, das seine eigene Länge ausgibt — ohne die Datei zu lesen.

1	Sei $f(p)$ das Programm „gib die Länge von p aus“.
2	f ist berechenbar: Es baut aus einem Text ein neues Programm.
3	Nach dem Rekursionssatz gibt es p mit: p verhält sich wie $f(p)$.
4	Also gibt p seine eigene Länge aus.

Der Beweis liefert kein Programm, sondern die Gewissheit, dass es eines gibt. Praktisch baut man es wie in Teil (a): Der Quine erzeugt seinen Text, und statt ihn auszugeben, misst man ihn.

Was sich damit alles bauen lässt.

Selbstkopierer	Gibt sich aus statt seine Länge — Kapitel 8
Selbstprüfer	Vergleicht seinen Text mit einer Prüfsumme
Selbst-optimierer	Erzeugt eine bessere Fassung seiner selbst
Der Gegenbeweis	**Tut das Gegenteil dessen, was ein Prüfer vorhersagt**

Die letzte Zeile ist Turings Beweis in einer Zeile. Der Rekursionssatz liefert das Programm, das den Prüfer widerlegt, und man muss es nicht einmal hinschreiben.

(c) Wann Selbstbezug widersprüchlich wird.
Die drei Zutaten.

Zutat	Beispiel	Ohne sie
Selbstbezug	„dieser Satz“	harmlos
Umkehrung	„ist falsch“	harmlos
Zweiwertigkeit	wahr oder falsch	harmlos

Alle drei zusammen ergeben den Widerspruch; je zwei sind ungefährlich. Das erklärt, warum es verschiedene Auswege gibt — man kann auf jede der drei Zutaten verzichten.

Die vier historischen Auswege.

Ausweg	Verzichtet auf	Preis
Typentheorie	Selbstbezug	umständlich
Mengenlehre (ZF)	unbeschränkte Bildung	kaum spürbar
Dreiwertige Logik	Zweiwertigkeit	schwächer
Gödels Weg	**auf nichts**	**Unvollständigkeit**

Die letzte Zeile ist die eigentliche Einsicht. Gödel verzichtete auf keine der drei Zutaten und ersetzte „ist falsch“ durch „ist unbeweisbar“. Damit wird aus dem Widerspruch eine wahre Aussage, die das System nicht beweisen kann — der Widerspruch wandert von der Wahrheit in die Beweisbarkeit.

Die Gegenüberstellung.

	„ich bin falsch“	„ich bin unbeweisbar“
Wenn wahr	dann falsch	dann unbeweisbar — passt
Wenn falsch	dann wahr	dann beweisbar, also wahr — passt nicht
Ergebnis	Widerspruch	**wahr und unbeweisbar**

Die zweite Spalte kippt in beiden Zeilen; die dritte kippt nur einmal und findet in der ersten Zeile Ruhe. Ein einziges ausgetauschtes Wort verwandelt eine Antinomie in einen Satz.

Wahrheit und Beweisbarkeit sind verschiedene Dinge

Wahr	Gilt in der gemeinten Struktur — etwa den natürlichen Zahlen
Beweisbar	Aus den Axiomen des Systems herleitbar
Gödels Satz	**Die beiden fallen auseinander**

Vor 1931 hoffte man, sie zur Deckung zu bringen: Ein vollständiges Axiomensystem hätte genau die wahren Sätze bewiesen. Gödel zeigte, dass das für die Arithmetik unmöglich ist — und dieser Riss ist der Gegenstand des nächsten Kapitels.

Russell[23] entdeckte die Antinomie; Quine[20] entwickelte den Kunstgriff, mit dem ein Satz ohne Zeigewort auf sich verweist; Kleene[14] bewies den Rekursionssatz und machte den Selbstbezug zum Werkzeug.

3 Universalität

Eine Maschine für alle Maschinen

Die Beobachtung	Ein Programm ist eine Zeichenkette und damit eine mögliche Eingabe
Die Folge	Es gibt eine Maschine, die jede andere nachahmt
Der Name	**universelle Turingmaschine**

Das klingt heute selbstverständlich — jeder Rechner führt beliebige Programme aus. 1936 war es eine Entdeckung: Man hatte Maschinen als feste *Apparate gedacht, und Turing zeigte, dass eine einzige Maschine genügt, wenn man die Bauanleitung als Eingabe zulässt.*

Der ganze Rest dieses Bandes hängt an diesem einen Schritt. Erst weil Programme Daten sind, kann ein Programm über Programme sprechen — und damit über sich selbst.

Wo die Universalität in der Reihe auftrat.

Band	Gestalt	Was Daten wird
2	universelle Maschine	**das Programm**
4	gespeichertes Programm	Befehle im Speicher
6	Übersetzer	**der Quelltext**
10	Abfragesprache	das Schema
12	Logik-programmierung	**die Regeln**
13	lernendes System	die Zielfunktion

Sechsmal derselbe Griff: Etwas, das bisher fest verdrahtet war, wird zur Eingabe. Jedes Mal gewinnt man Allgemeinheit — und jedes Mal handelt man sich die Grenze mit ein, die daraus folgt.

Die These von Church und Turing

Die Behauptung	Alles intuitiv Berechenbare ist Turing-berechenbar
Der Status	**kein Satz** — der eine Begriff ist nicht formal
Der Beleg	Alle vorgeschlagenen Modelle erwiesen sich als gleichmächtig

Modell	Jahr	Urheber
Rekursive Funktionen	1934	Gödel, Herbrand
Lambda-Kalkül	1936	Church
Turingmaschine	1936	Turing
Postsysteme	1943	Post
Registermaschinen	1963	Shepherdson, Sturgis
Zellularautomaten	1970	Conway

Sechs Modelle, aus völlig verschiedenen Richtungen entworfen, und alle berechnen genau dasselbe. Das ist der stärkste Beleg, den es für eine unbeweisbare These geben kann — und der Grund, warum sie allgemein akzeptiert ist.

Warum Universalität billig ist

Es braucht erstaunlich wenig, um universell zu werden.

System	Umfang	
Kleinste Turingmaschine	2 Zustände, 3 Zeichen	
Regel 110	**ein Zellularautomat**	eine Zeile Regel
Lambda-Kalkül	**drei Bildungsregeln**	
Conways Spiel des Lebens	vier Regeln	
Das Kartenspiel Magic	unabsichtlich	

Die letzte Zeile ist kein Scherz: Man hat gezeigt, dass sich in den Regeln eines Sammelkartenspiels eine Turingmaschine bauen lässt. Universalität stellt sich ein, sobald ein System Wiederholung, Verzweigung und unbegrenzten Speicher erlaubt — und das geschieht leicht aus Versehen.

Daraus folgt eine unangenehme Praxisregel: Wer eine Konfigurationssprache entwirft und sie „ein bisschen mächtiger“ macht, macht sie leicht unentscheidbar. Dann lässt sich nicht mehr prüfen, ob eine Konfiguration hält.

Aufgabe 3.1.

(a) Bauen Sie eine universelle Maschine im Kleinen.
(b) Prüfen Sie, wann ein System universell wird.
(c) Untersuchen Sie, was Universalität kostet.

Lösung Schritt für Schritt.

(a) Eine universelle Maschine im Kleinen.
Die Aufgabe. Ein Auswerter für einfache Registerbefehle, geschrieben in derselben Sprache.

Befehl	Wirkung
`INC r`	$r \leftarrow r + 1$
`DEC r`	$r \leftarrow r - 1$, falls > 0
`JZ r,m`	Sprung nach m, falls $r = 0$
`HALT`	Ende

Der Auswerter.

1	Kodiere das Programm als Zahlenfolge im Speicher.
2	Halte einen Befehlszähler und die Register.
3	Lies den Befehl, verzweige nach seinem Code, führe ihn aus.
4	Erhöhe den Zähler und wiederhole.

Größe	Wert	
Befehle des Auswerters	≈ 40	
Register	≈ 6	
Verlangsamung	$\approx$ **30**$\times$	je Schicht

Vierzig Befehle genügen, um jedes Programm dieser Sprache auszuführen — auch den Auswerter selbst. Damit ist die Sprache universell, und damit ist ihr Halteproblem unentscheidbar.

Der Auswerter im Auswerter.

Schichten	Verlang-samung	Bei 10^9 Schritten
0	1	1 s
1	30	30 s
2	900	15 min
3	27 000	7,5 h
5	$2{,}4 \cdot 10^7$	9 Monate

Die Verlangsamung ist multiplikativ und deshalb nach wenigen Schichten untragbar. Bemerkenswert: Sie ist ein konstanter *Faktor je Schicht, ändert also keine Komplexitätsklasse. Universalität kostet Geschwindigkeit, nicht Berechenbarkeit.*

(b) Wann ein System universell wird.

System	Universell?	Warum
Aussagenlogik	nein	endlich viele Belegungen
Reguläre Ausdrücke	nein	kein Speicher
Kontextfreie Grammatiken	nein	nur Kellerspeicher
Tabellenkalkulation ohne Zyklen	nein	
Mit Zyklen	**ja**	unentscheidbar
Prädikatenlogik	ja	**Band 1**

Die fünfte Zeile ist die praktisch wichtigste. Eine Tabellenkalkulation mit erlaubten Zyklen und iterativer Auswertung ist universell — und damit lässt sich nicht mehr entscheiden, ob eine Berechnung terminiert. Dieselbe Schwelle trennt Konfigurationsdateien von Programmiersprachen.

Die drei Zutaten.

Zutat	Fehlt sie	Folge
Verzweigung	keine Fallunterscheidung	sehr schwach
Wiederholung	feste Schrittzahl	stets terminierend
Unbegrenzter Speicher	endlicher Automat	entscheidbar

Die zweite Zeile beschreibt die primitiv rekursiven Funktionen: Man darf beliebig oft wiederholen, muss aber die Zahl der Durchläufe vorher kennen. Solche Programme halten immer — und trotzdem gibt es berechenbare Funktionen, die sie nicht erreichen.

(c) Was Universalität kostet.

Gewinn	Preis	
Ein Gerät für alles	Geschwindigkeit	konstanter Faktor
Programme als Daten	**Halteproblem**	unentscheidbar
Selbstbezug verfügbar	**Unvollständigkeit**	Kap. 4
Erweiterbarkeit	keine vollständige Prüfung	Rice

Die Tabelle ist die These dieses Bandes in vier Zeilen. Jede Zeile der linken Spalte ist ein Grund, warum man Universalität will; jede Zeile der rechten ist ihr unvermeidlicher Begleiter.

Der Verzicht als Entwurfsentscheidung.

System	Verzichtet auf	Gewinnt
SQL (Kernsprache)	Rekursion ohne Grenze	Auswertbarkeit
Reguläre Ausdrücke	Speicher	lineare Zeit
Typsysteme	volle Ausdruckskraft	Entscheidbarkeit
Beweisassistenten	unbeschränkte Rekursion	Terminierung

Vier bewusste Verzichte, alle aus demselben Grund: Man will etwas garantieren können. Das ist die praktische Lehre dieses Bandes — Universalität ist kein Ziel, sondern eine Abwägung. Wer prüfen will, muss auf Mächtigkeit verzichten.

Universalität schleicht sich ein

Das Muster	Eine einfache Sprache wird um „nur eine Kleinigkeit“ erweitert
Die Kleinigkeit	Templates, Makros, Zeiger, Verweise auf Verweise
Die Folge	Das System ist universell und damit unprüfbar

Mehrere weit verbreitete Konfigurations- und Auszeichnungssprachen sind auf diesem Weg versehentlich universell geworden. Danach lässt sich nicht mehr garantieren, dass die Verarbeitung einer Eingabe terminiert — und das ist zugleich eine Sicherheitslücke.

Turing[27] konstruierte die universelle Maschine; Church[5] kam von einer anderen Seite zum selben Begriff; Davis[7] versammelt die Originalarbeiten dieser Jahre.

Part II

Die Grenzen

4 Gödel

Die beiden Sätze

Erster Satz	Jedes widerspruchsfreie System, das die Arithmetik umfasst und dessen Axiome aufzählbar sind, ist **unvollständig**
Zweiter Satz	Ein solches System kann seine **eigene Widerspruchsfreiheit nicht beweisen**

Die Voraussetzungen sind wichtiger als das Ergebnis. Drei Bedingungen müssen erfüllt sein — Widerspruchsfreiheit, genug Arithmetik, aufzählbare Axiome. Fällt eine davon weg, gilt der Satz nicht, und es gibt tatsächlich vollständige Systeme.

System	Vollständig?	Warum
Aussagenlogik	ja	**keine Arithmetik**
Presburger-Arithmetik	ja	**keine Multiplikation**
Theorie der reellen Körper	ja	Tarski 1951
Euklidische Geometrie	ja	
Peano-Arithmetik	**nein**	**Gödel**
Mengenlehre	nein	

Die zweite Zeile ist besonders lehrreich. Addition allein reicht nicht für Unvollständigkeit; erst mit der Multiplikation lassen sich Zeichenketten kodieren, und erst dann kann das System über sich selbst sprechen. Die Grenze liegt genau dort, wo die Selbstbeschreibung beginnt.

Der Beweis in fünf Schritten

1	**Kodierung**: Jeder Formel und jedem Beweis wird eine Zahl zugeordnet.
2	**Arithmetisierung**: „b ist ein Beweis von f“ wird zu einer arithmetischen Aussage.
3	**Beweisbarkeit**: $\mathrm{Bew}(f)$ heißt „es gibt ein b, das f beweist“.
4	**Selbstbezug**: Konstruiere G mit $G \leftrightarrow \neg\mathrm{Bew}(\ulcorner G \urcorner)$.
5	**Schluss**: Wäre G beweisbar, wäre das System widersprüchlich.

Schritt 1 ist die eigentliche Erfindung. Gödel zeigte, wie sich Syntax in Arithmetik übersetzen lässt — mit Primzahlpotenzen. Damit wird eine Aussage über Formeln zu einer Aussage über Zahlen, und die Arithmetik kann über sich selbst sprechen.

Aus heutiger Sicht ist Schritt 1 selbstverständlich: Jeder Rechner speichert Programme als Zahlen. 1931 gab es keine Rechner, und die Idee, Syntax zu arithmetisieren, war neu.

Die Kodierung

Jedem Zeichen wird eine Nummer zugeordnet, jeder Zeichenkette das Produkt von Primzahlpotenzen.

Zeichen	Nr.	Primzahl	Beitrag
$\neg$	1	2	2^1
$\vee$	2	3	3^2
0	3	5	5^3

Die Zerlegung in Primfaktoren ist eindeutig, also lässt sich aus der Zahl die Zeichenkette zurückgewinnen. Das ist der ganze Kunstgriff — und er verbraucht astronomisch große Zahlen, was für den Beweis gleichgültig ist.

Aufgabe 4.1.

(a) Kodieren Sie eine Formel als Zahl.

(b) Führen Sie das Argument des ersten Satzes durch.

(c) Untersuchen Sie, was aus Gödels Sätzen nicht folgt.

Lösung Schritt für Schritt.

(a) Eine Formel kodieren.
Die Formel. $\neg 0$ — also die Zeichenfolge $(\neg, 0)$ mit den Nummern $(1, 3)$.

$$\ulcorner \neg 0 \urcorner = 2^1 \cdot 3^3 = 2 \cdot 27 = \mathbf{54}.$$

Die Rückgewinnung.

Primzahl	Exponent	Zeichen-Nr.	Zeichen
2	1	1	¬
3	3	3	0

Aus $54 = 2^1 \cdot 3^3$ *liest man die Folge* $(1, 3)$ *ab, und daraus die Formel. Die Zuordnung ist umkehrbar, und genau das braucht der Beweis.*

Das Größenwachstum.

Zeichen	Kodezahl	Stellen	
2	54	2	
5	$\approx 10^{10}$	10	
20	$\approx 10^{80}$	80	
100	$\approx 10^{700}$	700	unhandlich

Für einen Beweis ist die Größe gleichgültig — die Zahl existiert, und das genügt. Praktisch benutzt man heute andere Kodierungen, etwa die Bytefolge einer Datei. Der Beweis funktioniert mit jeder umkehrbaren Zuordnung.

(b) Das Argument des ersten Satzes.
Der Satz G**.** Er besagt: „Es gibt keinen Beweis für die Formel mit der Nummer $\ulcorner G \urcorner$."
Fall 1: G **ist beweisbar.**

1	Dann gibt es einen Beweis für G.
2	G behauptet aber, es gebe keinen.
3	Das System beweist eine falsche Aussage.
4	**Also ist das System widersprüchlich** — ausgeschlossen.

Fall 2: $\neg G$ ist beweisbar.

1	$\neg G$ besagt: es gibt einen Beweis für G.
2	Ist das wahr, so ist G beweisbar — Fall 1, Widerspruch.
3	Ist es falsch, so beweist das System eine falsche Aussage.
4	**Wieder widersprüchlich.**

	beweisbar?	**wahr?**
G	nein	**ja**
$\neg G$	nein	nein

Weder G noch seine Verneinung ist beweisbar — das ist die Unvollständigkeit. Und G ist wahr*: Es behauptet, unbeweisbar zu sein, und ist es. Diese Wahrheit erkennt man von außen, indem man über das System spricht, nicht in ihm.*

Der übliche Einwand.

Der Einwand	„Dann nehme man G als neues Axiom hinzu“
Die Antwort	Das neue System ist wieder aufzählbar, also greift der Satz erneut
Die Folge	Es entsteht ein **neues** G' — und so fort

Man kann beliebig viele solcher Sätze hinzufügen, sogar unendlich viele, und erhält stets ein unvollständiges System. Der Satz ist nicht durch Nachbessern zu umgehen, und darin liegt seine Schärfe.

(c) Was nicht folgt.

Behauptung	Urteil	Berichtigung
Es gibt absolut unbeweisbare Wahrheiten	falsch	**nur relativ zum System**
Mathematik ist unvollendbar	teils	für die Arithmetik
Kein System versteht sich selbst	falsch	**nur nicht vollständig**
Der Mensch übertrifft Maschinen	unbelegt	Kapitel 10
Wahrheit ist relativ	falsch	

Die erste Zeile im Detail.

G **in PA**	unbeweisbar
G **in PA** $+ G$	trivial beweisbar
G **in ZF**	beweisbar — die Mengenlehre ist stärker

Gödels Satz für die Peano-Arithmetik lässt sich in der Mengenlehre beweisen. Es gibt also keine Aussage, die überhaupt *unbeweisbar wäre — nur solche, die in einem* bestimmten *System unbeweisbar sind. Die verbreitete Fassung lässt genau dieses Wort weg.*

Ein Beispiel, das keine Konstruktion ist.

Aussage	in PA	Bemerkung
Gödelsatz G	unbeweisbar	künstlich
Goodstein-Satz	**unbeweisbar**	**natürlich**
Satz von Paris-Harrington	unbeweisbar	kombinatorisch

Der häufigste Einwand gegen Gödel lautet, G sei eigens konstruiert und komme in der Mathematik nicht vor. Die zweite Zeile widerlegt ihn: Der Goodstein-Satz ist eine harmlose Aussage über Zahlenfolgen, wahr, und in der Peano-Arithmetik unbeweisbar.

Solche Beispiele wurden erst in den 1970er Jahren gefunden, vierzig Jahre nach Gödel. Bis dahin konnte man tatsächlich glauben, die Unvollständigkeit betreffe nur eigens gebastelte Sätze.

Der zweite Satz trifft die Praxis härter

Die Aussage	Ein System kann seine eigene Widerspruchsfreiheit nicht beweisen
Die Folge	Wer einem Beweisassistenten trauen will, braucht ein stärkeres System
Und dann?	**Dessen Widerspruchsfreiheit ist wieder offen**

Es gibt keinen Boden, auf dem man endgültig stehen kann. Die Widerspruchsfreiheit der Arithmetik wurde 1936 bewiesen — mit einem Mittel, das über die Arithmetik hinausgeht, und dessen eigene Widerspruchsfreiheit ebenso offen ist. Das ist kein Skandal, sondern die Lage.

Praktisch geht man damit um wie mit dem Halteproblem: Man arbeitet mit Systemen, an die man aus Erfahrung glaubt, und macht die Annahme explizit. Kapitel 7 zeigt dieselbe Struktur beim Vertrauen in Übersetzer.

Gödel[10] legte beide Sätze 1931 vor; Nagel und Newman[16] erklären sie ohne Formalismus und ohne Verfälschung; Smullyan[25] bringt die technisch saubere Darstellung mit allen Voraussetzungen.

5 Turing

Das Halteproblem

Die Frage	Gibt es ein Programm H, das für jedes Paar (p, e) entscheidet, ob p auf e hält?
Die Antwort	**Nein**
Der Beweis	Vier Zeilen, mit dem Rekursionssatz aus Kapitel 2

Der Beweis.

1	Angenommen, H existiert.
2	Baue D: „Sei s mein eigener Quelltext. Wenn $H(s, s)$ sagt ‚hält', laufe ewig; sonst halte."
3	Was tut D auf sich selbst?
4	Hält D, so läuft D ewig. Läuft D ewig, so hält D. **Widerspruch.**

Der Widerspruch trifft nicht D — das Programm ist völlig unproblematisch, sein Bau ist durch den Rekursionssatz gesichert. Er trifft die Annahme, dass H existiert. Das ist der Unterschied zu Cantor, wo die Aufzählung fällt.

Und es ist genau dieselbe Diagonale: D verhält sich auf sich selbst anders, als H vorhersagt. Der Index läuft über Programme, die diagonale Stelle ist die Selbstanwendung, und die Umkehrung steckt im Wort „sonst".

Der Satz von Rice

Die Aussage	**Jede nichttriviale Eigenschaft des Verhaltens eines Programms ist unentscheidbar**
Nichttrivial	Manche Programme haben sie, andere nicht
Verhalten	Was das Programm berechnet, nicht wie es aussieht

Das ist erheblich stärker als das Halteproblem und wird viel zu selten genannt. Es besagt: Man kann von einem Programm im Allgemeinen gar nichts *Interessantes über sein Verhalten entscheiden.*

Frage	**entscheidbar?**	
Hält das Programm?	nein	Turing
Gibt es je 0 aus?	nein	Rice
Berechnet es eine konstante Funktion?	nein	Rice
Sind zwei Programme gleichwertig?	nein	Rice
Greift es je auf Adresse 0 zu?	nein	Rice
Hat es mehr als 100 Zeilen?	ja	**keine Verhaltensfrage**
Enthält es das Wort `goto`?	ja	**syntaktisch**
Hält es in $\leq 10^6$ Schritten?	ja	**beschränkt**

Die untere Hälfte zeigt die beiden Auswege. Man fragt nach der Gestalt statt nach dem Verhalten, oder man setzt eine Schranke. Beides tun Werkzeuge in der Praxis ständig, und beides funktioniert.

Unentscheidbar heißt nicht hoffnungslos

Ausweg	Preis	Beispiel
Dritte Antwort	„weiß nicht“	Programmprüfer
Nur eine Richtung	Fehlalarme	**Typprüfung**
Schranke setzen	Unvollständig	Zeitlimit
Sprache schwächen	Ausdruckskraft	Terminierungs-prüfer

Die zweite Zeile ist die häufigste. Ein Typsystem lehnt manche korrekten Programme ab und akzeptiert kein fehlerhaftes — es irrt also nur in eine Richtung. Das ist der übliche Umgang mit Unentscheidbarkeit und hat sich hervorragend bewährt.

Aufgabe 5.1.

(a) Führen Sie den Halteproblembeweis vollständig durch.
(b) Zeigen Sie eine Unentscheidbarkeit durch Reduktion.
(c) Untersuchen Sie, wie Werkzeuge damit umgehen.

Lösung Schritt für Schritt.

(a) Das Halteproblem.
Die Annahme. Ein Programm `haelt(p, e)` liefert stets `true` oder `false` und irrt nie.
Das Gegenprogramm.

```
def D(s):
if haelt(s, s):
while True:  pass
else:
return
```

Die Fallunterscheidung für `D(D)`.

Annahme	`haelt(D,D)`	**D tut**	**Also**
D hält	`true`	**läuft ewig**	hält nicht
D hält nicht	`false`	**hält**	hält

Beide Zeilen enden im Gegenteil ihrer Annahme. Da D ohne Weiteres schreibbar ist — es benutzt nur `haelt` und eine Fallunterscheidung — muss `haelt` selbst der Fehler sein. □

Die Diagonale sichtbar gemacht.

	auf P_1	**auf** P_2	**auf** P_3	…
P_1	**h**	h	∞	
P_2	∞	**h**	h	
P_3	h	∞	∞	
D	∞	∞	**h**	

D tut in Spalte n das Gegenteil dessen, was P_n dort tut. Also ist D keines der P_n — was unmöglich ist, denn D ist ein Programm und alle Programme sind aufgezählt. Es ist Cantors Tabelle, mit Programmen statt Folgen.

(b) Eine Reduktion.
Die Frage. Ist entscheidbar, ob ein Programm je die Zahl 42 ausgibt?
Die Reduktion. Angenommen, ein Prüfer `gibt42(q)` existiert.

1	Gegeben (p, e), baue q: „führe p auf e aus, danach gib 42 aus“.
2	q gibt genau dann 42 aus, wenn p auf e hält.
3	Also entscheidet `gibt42(q)` das Halteproblem.
4	**Widerspruch** — also gibt es `gibt42` nicht.

Das ist das Standardmuster. Man zeigt nicht direkt, dass eine Frage unentscheidbar ist, sondern dass ihre Lösung das Halteproblem lösen würde. Der Bau von q aus (p, e) ist rein mechanisch — genau das verlangt eine Reduktion.

Weitere Reduktionen nach demselben Muster.

Frage	Baue q so	
Ist q die leere Funktion?	p ausführen, dann 1 liefern	unentscheidbar
Greift q auf Adresse 0 zu?	p ausführen, dann zugreifen	unentscheidbar
Ist q gleichwertig zu `nichts`?	wie oben	unentscheidbar

Der Bauplan ist stets derselbe: erst p laufen lassen, dann die fragliche Eigenschaft herbeiführen. Damit hängt die Eigenschaft am Halten von p, und Rice' Satz ist im Grunde die Verallgemeinerung dieses einen Kniffs.

(c) Wie Werkzeuge damit umgehen.
Ein Terminierungsprüfer in der Praxis.

Antwort	Anteil	Nutzen
„terminiert“	$\approx 78\,\%$	bewiesen
„terminiert nicht“	$\approx 9\,\%$	Gegenbeispiel
„weiß nicht“	$\approx 13\,\%$	nichts

Siebenundachtzig Prozent der Fälle werden entschieden, und die restlichen dreizehn bleiben offen. Das Halteproblem ist unentscheidbar, und trotzdem ist ein solches Werkzeug nützlich — weil die Programme, die Menschen schreiben, keine Diagonalkonstruktionen sind.

Die beiden Irrtumsarten.

	Fehlalarm	Übersehener Fehler
Typprüfer	möglich	ausgeschlossen
Testverfahren	ausgeschlossen	möglich
Vollständig	**unmöglich — Rice**	

Man kann in einer Richtung sicher sein, nie in beiden. Ein Typprüfer lehnt manche guten Programme ab; ein Test findet nicht alle Fehler. Wer beides zugleich verspricht, hat entweder die Sprache eingeschränkt oder irrt sich.

Warum die Praxis trotzdem funktioniert.

Der Grund	Unentscheidbarkeit ist eine Aussage über *alle* Programme
Die Praxis	Menschen schreiben strukturierte Schleifen mit erkennbaren Maßen
Die Folge	Die schwierigen Fälle kommen fast nie vor

Das ist dieselbe Lage wie bei NP-Vollständigkeit in Band 3: Die schlimmsten Fälle existieren, und die tatsächlich auftretenden sind harmlos. Eine Grenze im schlimmsten Fall sagt wenig über den Alltag — und wird trotzdem regelmäßig so gelesen.

Der Ausweg über die Sprache

Sprache	Terminierung	
Allgemeine Sprachen	unentscheidbar	
Primitiv rekursiv	garantiert	nicht universell
Beweisassistenten	garantiert	Rekursion muss abnehmen

Beweisassistenten verlangen, dass jede Rekursion nachweislich auf ein kleineres Argument geht. Damit terminiert jedes Programm — und die Sprache ist nicht mehr universell. Man hat die Grenze weggekauft, indem man die Mächtigkeit verkauft hat. Das ist die Abwägung aus Kapitel 3, hier bewusst getroffen.

Turing[27] bewies die Unentscheidbarkeit; Rice[21] verallgemeinerte sie auf alle nichttrivialen Verhaltenseigenschaften; Post[19] untersuchte, wie sich unlösbare Probleme nach Schwierigkeit ordnen lassen.

6 Wo die Diagonale versagt

Die Diagonale in der Komplexitätstheorie

Dasselbe Verfahren beweist, dass mehr Zeit mehr leistet.

Zeithierarchie	Es gibt Probleme in Zeit n^3, die nicht in Zeit n^2 lösbar sind
Der Beweis	Diagonalisierung über alle Maschinen mit der kleineren Schranke
Der Kunstgriff	**Die Diagonalmaschine simuliert und kehrt um**

Man baut eine Maschine, die auf Eingabe i die i-te Maschine für n^2 Schritte simuliert und dann das Gegenteil ausgibt. Sie braucht etwas mehr als n^2 Zeit, weil das Simulieren kostet — deshalb der Abstand zwischen den Schranken.

Satz	Bewiesen	Mit
$\mathrm{P} \subsetneq \mathrm{EXPTIME}$	ja	**Diagonale**
$\mathrm{L} \subsetneq \mathrm{PSPACE}$	ja	Diagonale
$\mathrm{P} \neq \mathrm{NP}$	**offen**	Diagonale genügt nicht

Die ersten beiden Zeilen sind seit 1965 bekannt und werden mit demselben Verfahren bewiesen, das dieser Band von Anfang an verfolgt. Die dritte ist die berühmteste offene Frage der Informatik — und der Grund für ihre Hartnäckigkeit ist der Gegenstand dieses Kapitels.

Warum die Diagonale bei P gegen NP versagt

Baker, Gill und Solovay zeigten 1975 ein Ergebnis über Beweise statt über Probleme.

Die Beobachtung	Diagonalisierung funktioniert auch dann, wenn alle Maschinen ein Orakel benutzen
Die Konstruktion	Es gibt ein Orakel A mit $\mathrm{P}^A = \mathrm{NP}^A$ und ein B mit $\mathrm{P}^B \neq \mathrm{NP}^B$
Der Schluss	**Kein Beweis, der Orakel überlebt, kann die Frage entscheiden**

Das ist ein Ergebnis über Beweismethoden, nicht über Komplexitätsklassen — und es ist selbst mit einer Diagonalkonstruktion bewiesen. Die Diagonale zeigt hier ihre eigene Grenze auf, was für diesen Band eine passende Pointe ist.

Was Orakel überlebt	Jedes Argument, das die Maschine nur als Schwarzen Kasten behandelt
Was nötig wäre	Ein Argument, das in die Maschine hineinschaut
Der Name	Nichtrelativierende Beweise

Diagonalisierung schaut nicht hinein — sie simuliert nur und kehrt um. Genau deshalb überlebt sie Orakel, und genau deshalb kann sie P gegen NP nicht entscheiden.

Zwei weitere Sperren

Nach 1975 fand man zwei weitere Klassen von Beweisen, die nicht ausreichen.

Jahr	Sperre	Was ausgeschlossen wird
1975	Relativierung	**Diagonalisierung**
1994	Natürliche Beweise	**kombinatorische Maße**
2008	Algebrisierung	algebraische Erweiterungen

Drei Sperren, die zusammen fast alle bekannten Beweistechniken ausschließen. Man weiß also ziemlich genau, wie ein Beweis nicht *aussehen kann — und hat trotzdem keine Vorstellung, wie er aussehen müsste.*

Das ist eine ungewöhnliche Lage in der Mathematik: Es gibt eine Metatheorie darüber, warum ein Problem widersteht. Sie erklärt fünfzig Jahre Erfolglosigkeit besser als jede Vermutung über den Schwierigkeitsgrad.

Aufgabe 6.1.

(a) Führen Sie den Zeithierarchiesatz durch.
(b) Bauen Sie ein Orakel, unter dem P gleich NP ist.
(c) Untersuchen Sie, was das für die Suche nach Beweisen bedeutet.

Lösung Schritt für Schritt.

(a) Der Zeithierarchiesatz.
Die Behauptung. Es gibt eine Sprache in $\mathrm{DTIME}(n^3)$, die nicht in $\mathrm{DTIME}(n^2)$ liegt.
Die Diagonalmaschine D**.**

1	Lies die Eingabe w; fasse sie als Kodierung einer Maschine M_w auf.
2	Simuliere M_w auf w für höchstens $\lvert w\rvert^2$ Schritte.
3	Hält M_w und akzeptiert, so **verwirf**.
4	Sonst **akzeptiere**.

Die Laufzeit.

Anteil	Kosten	
Simulierte Schritte	n^2	
Kosten je Schritt	$\mathcal{O}(\log n)$	**Bandverwaltung**
Gesamt	$\mathcal{O}(n^2 \log n)$	$\subseteq \mathcal{O}(n^3)$

Der Widerspruch.

1	Angenommen, eine Maschine M in $\mathcal{O}(n^2)$ entscheidet dieselbe Sprache wie D.
2	Sei w eine Kodierung von M, lang genug, dass die Simulation reicht.
3	Dann tut D auf w das Gegenteil von M auf w.
4	Aber D und M sollen übereinstimmen. **Widerspruch.**

Es ist wörtlich derselbe Beweis wie beim Halteproblem, nur mit einer Zeitschranke im Simulationsschritt. Der Zusatzaufwand des Simulierens — der Faktor $\log n$ — ist der Grund, warum zwischen den beiden Schranken ein Abstand bleiben muss.

Schranken	Trennung?	
n^2 gegen n^3	ja	Abstand genügt
n^2 gegen $n^2 \log^2 n$	ja	knapp
n^2 gegen n^2	nein	kein Platz für Simulation

(b) Ein Orakel mit $\mathbf{P}^A = \mathbf{NP}^A$.
Die Wahl. A sei eine PSPACE-vollständige Sprache, etwa die Auswertung quantifizierter Boolescher Formeln.

1	Mit dem Orakel A kann eine Polynomzeitmaschine jede PSPACE-Frage stellen.
2	Also $\text{PSPACE} \subseteq \text{P}^A$.
3	Umgekehrt: $\text{NP}^A \subseteq \text{PSPACE}$, denn man kann alle Zweige nacheinander durchgehen und das Orakel selbst berechnen.
4	Zusammen: $\text{NP}^A \subseteq \text{PSPACE} \subseteq \text{P}^A \subseteq \text{NP}^A$.

Alle Inklusionen sind Gleichheiten, also $P^A = NP^A$. □

Ein Orakel mit $\text{P}^B \neq \text{NP}^B$.

1	L_B sei: „B enthält ein Wort der Länge n“.
2	L_B liegt in NP^B: rate das Wort, frage das Orakel.
3	Baue B schrittweise, sodass jede Polynomzeitmaschine irrt.
4	Bei n Stellen und n^k Anfragen bleiben **ungefragte Wörter übrig**.
5	Dort setzt man das Orakel gegen die Maschine.

n	**Wörter** 2^n	**Anfragen** n^{10}	**Rest**
20	10^6	$1{,}0 \cdot 10^{13}$	0
60	$1{,}2 \cdot 10^{18}$	$6{,}0 \cdot 10^{17}$	> 0
100	$1{,}3 \cdot 10^{30}$	10^{20}	sehr viele
200	$1{,}6 \cdot 10^{60}$	10^{23}	fast alle

Ab einer gewissen Länge wächst 2^n schneller als jedes Polynom, und dann bleiben Wörter übrig, die die Maschine nie gefragt hat. Genau dort legt man das Orakel so fest, dass die Maschine sich irrt — wieder eine Diagonalisierung, diesmal über Maschinen und Orakelstellen zugleich.

(c) Was das für die Beweissuche bedeutet.

Die Lage	Zwei Orakel, unter denen die Antwort verschieden ausfällt
Die Folge	Jeder Beweis muss auf Eigenschaften zurückgreifen, die Orakel zerstören
Konkret	Er muss die *innere Struktur* der Berechnung ausnutzen

Methode	**Relativiert?**	**Also brauchbar?**
Diagonalisierung	ja	nein
Simulation	ja	nein
Reduktion	ja	nein
Schaltkreis-untergrenzen	nein	vielleicht
Interaktive Beweise	nein	**haben schon gewirkt**

Die letzte Zeile ist ein Hoffnungsschimmer. In den 1990er Jahren wurde mit interaktiven Beweisen ein Ergebnis erzielt, das nicht *relativiert — es gilt also, ohne Orakel zu überleben. Damit ist gezeigt, dass es solche Methoden gibt.*

Für P gegen NP hat das bisher nicht gereicht, denn die Sperre von 1994 schließt zusätzlich die naheliegenden Schaltkreisargumente aus. Man steht zwischen drei Verboten und sucht nach einem vierten Weg.

Was dieses Kapitel für den ganzen Band bedeutet

Die Diagonale	Beantwortet Fragen der **Berechenbarkeit** vollständig
Sie versagt	Bei Fragen der **Effizienz**
Der Grund	Sie unterscheidet nicht, *wie* eine Maschine rechnet, nur *was* sie ausgibt

Das ist die wichtigste Einschränkung der These dieses Bandes. Selbstbezug erklärt, warum manche Fragen unentscheidbar sind. Er erklärt nicht, warum manche entscheidbaren Fragen teuer sind — und das ist die Frage, an der die Informatik seit fünfzig Jahren arbeitet.

Man kann es auch anders lesen: Die Grenze der Berechenbarkeit ist verstanden, die Grenze der Effizienz nicht. Und wo der seltsame Kreis nicht greift, hat man bislang gar kein Werkzeug.

Hartmanis und Stearns[11] bewiesen die Hierarchiesätze; Baker, Gill und Solovay[2] zeigten die Grenze der Methode; Cook[6] führte den Begriff der NP-Vollständigkeit ein, um den es dabei geht.

Part III

Der Kreis in der Technik

7 Der Übersetzer, der sich selbst übersetzt

Das Henne-Ei-Problem

Die Lage	Ein Übersetzer für C ist selbst in C geschrieben
Die Frage	Womit wurde er übersetzt?
Die Antwort	Mit einer früheren Fassung — und ganz am Anfang von Hand

Das ist der praktischste seltsame Kreis der ganzen Informatik, und er wird täglich durchlaufen. Jeder große Übersetzer der Welt ist das Ergebnis einer Kette, die vor Jahrzehnten mit einem von Hand geschriebenen Programm begann.

Das Vorgehen.

1	Schreibe einen einfachen Übersetzer für eine Teilsprache, in Assembler.
2	Schreibe den vollen Übersetzer in dieser Teilsprache.
3	Übersetze ihn mit Stufe 1.
4	**Übersetze ihn nun mit sich selbst.**
5	Wirf Stufe 1 weg.

Nach Schritt 5 gibt es kein Programm mehr, das den Übersetzer erzeugen könnte, außer ihm selbst. Der Kreis hat sich geschlossen, und die Leiter, über die man hinaufgestiegen ist, wurde weggeworfen.

Warum Schritt 4 nötig ist

Erzeugt durch	Ergebnis	
Stufe 1	**korrekt, aber langsam**	Stufe 1 optimiert nicht
Sich selbst	**korrekt und schnell**	
Nochmals	**bitgleich**	**die Probe**

Die dritte Zeile ist eine der elegantesten Prüfungen der Softwaretechnik. Übersetzt der Übersetzer sich zweimal, so müssen die beiden Ergebnisse Byte für Byte gleich sein — denn beide Male lief dieselbe Übersetzung auf demselben Quelltext. Weicht etwas ab, liegt ein Fehler vor.

Thompsons Angriff

KEN THOMPSON beschrieb 1984 in seiner Turing-Award-Rede einen Angriff, der den Kreis ausnutzt.

1	Baue in den Übersetzer eine Regel: „Übersetzt du das Anmeldeprogramm, so füge eine Hintertür ein.“
2	Baue eine zweite Regel: „Übersetzt du **den Übersetzer**, so füge beide Regeln wieder ein.“
3	Übersetze den Übersetzer damit.
4	**Entferne beide Regeln aus dem Quelltext.**

Nach Schritt 4 steht im Quelltext des Übersetzers nichts Verdächtiges mehr. Trotzdem baut jede neu übersetzte Fassung die Hintertür wieder ein — und den Mechanismus, der sie einbaut. Der Angriff lebt im Binärprogramm und pflanzt sich über Generationen fort.

Prüfung	Findet es?	
Quelltext des Anmeldeprogramms	nein	sauber
Quelltext des Übersetzers	nein	sauber
Neu übersetzen	nein	reproduziert sich
Binärprogramm zerlegen	ja	sehr mühsam

Thompsons Schluss: Man kann keinem Programm trauen, das man nicht selbst vollständig erzeugt hat — einschließlich der Werkzeuge, mit denen man es erzeugt. Das ist die Struktur des zweiten Gödelsatzes, in Werkzeugform.

Es gibt eine Abhilfe, aber keine vollständige

Die Idee	Übersetze den Quelltext mit einem *zweiten*, unabhängigen Übersetzer
Der Test	Beide Ergebnisse müssen bei nochmaliger Selbstübersetzung übereinstimmen
Die Grenze	**Beide könnten denselben Angriff tragen**

Das Verfahren heißt Gegenprüfung durch doppeltes Übersetzen und wurde 2009 ausgearbeitet. Es funktioniert, solange die beiden Übersetzer keinen gemeinsamen Vorfahren haben — und da fast alle Übersetzer der Welt eine gemeinsame Ahnenreihe haben, ist das eine ernsthafte Einschränkung.

Aufgabe 7.1.

(a) Verfolgen Sie ein Bootstrapping über mehrere Stufen.
(b) Rechnen Sie Thompsons Angriff durch.
(c) Untersuchen Sie die Gegenprüfung.

Lösung Schritt für Schritt.

(a) Bootstrapping über mehrere Stufen.

Die Lage. Eine neue Sprache L soll einen Übersetzer in L erhalten.

Stufe	Geschrieben in	Übersetzt von	Kann
0	Assembler	Hand	L_0, winzig
1	L_0	Stufe 0	L_1
2	L_1	Stufe 1	L_2
3	L_2	Stufe 2	**L ganz**
3′	L	**Stufe** 3	L, optimiert

Die Größen.

Stufe	Zeilen	Sprachumfang	Optimierung
0	800	5 %	keine
1	3000	25 %	keine
2	12 000	70 %	einfach
3	**60 000**	**100** %	voll

Die Zahl der von Hand geschriebenen Zeilen ist erstaunlich klein: achthundert. Alles Weitere entsteht durch wiederholte Selbstanwendung, und jede Stufe kann in einer bequemeren Sprache geschrieben werden als die vorige.

Der Fixpunkt.

Durchlauf	Erzeugt von	Größe	Gleich?
C_1	Stufe 2	2,1 MB	—
C_2	C_1	1,8 MB	nein
C_3	C_2	1,8 MB	**ja**
C_4	C_3	1,8 MB	ja

Ab C_3 ändert sich nichts mehr: Der Übersetzer ist ein Fixpunkt seiner selbst. C_1 war kleiner optimiert, weil Stufe 2 es nicht besser konnte; C_2 ist bereits vom vollen Übersetzer erzeugt. Diese Dreistufenprobe ist Standard bei jedem großen Übersetzerprojekt.

(b) Thompsons Angriff.
Der Übersetzer im Pseudocode.

```
def uebersetze(quelle):
if istAnmeldung(quelle):
return kompiliere(quelle + HINTERTUER)
if istUebersetzer(quelle):
return kompiliere(quelle + DIESE_ZWEI_REGELN)
return kompiliere(quelle)
```

Die entscheidende Zeile.

Zeile 5	Fügt beide Regeln in jeden neu übersetzten Übersetzer ein
Wie	Mit dem Rekursionssatz — **das Programm kennt seinen eigenen Text**
Kapitel 2	Genau der Quine-Kunstgriff, hier als Waffe

Der Verlauf über die Generationen.

Fassung	Quelltext	Binärprogramm	Hintertür
V_0	verseucht	verseucht	ja
V_1	sauber	verseucht	ja
V_2	sauber	verseucht	ja
V_{100}	sauber	**verseucht**	**ja**

Ab V_1 ist jeder Quelltext sauber, und trotzdem trägt jede Fassung die Hintertür. Man kann den gesamten Quelltext aller beteiligten Programme prüfen und findet nichts. Der Angriff sitzt in einer Erblinie, nicht in einem Text.

Thompsons Rede ist deshalb eine der meistzitierten der Informatik. Sie zeigt, dass das Prüfen von Quelltext prinzipiell nicht genügt — und dass die Ursache nicht Nachlässigkeit ist, sondern die Selbstbezüglichkeit des Werkzeugs.

(c) Die Gegenprüfung.
Das Verfahren. Gegeben: Quelltext S, verdächtiges Binärprogramm A, unabhängiger Übersetzer B.

1	$X = A(S)$ — der verdächtige Übersetzer übersetzt seinen Quelltext.
2	$Y = B(S)$ — der unabhängige tut dasselbe.
3	X und Y sind vermutlich verschieden (andere Optimierung).
4	**Aber**: $X(S)$ und $Y(S)$ müssen **bitgleich** sein.

Schritt 4 ist der Kern. X und Y sind zwei verschiedene Binärprogramme, die beide dieselbe Übersetzungsvorschrift S umsetzen. Wendet man beide auf S an, so muss dasselbe herauskommen — denn das Ergebnis hängt nur von S ab, nicht davon, wie S selbst übersetzt wurde.

Fall	$X(S)$	$Y(S)$	**Schluss**
A sauber	Z	Z	gleich
A verseucht	Z'	Z	**verschieden**

Ein verseuchtes A erzeugt ein X, das die Regeln trägt; also fügt X sie auch Z' hinzu. Y ist sauber und erzeugt reines Z. Der Unterschied fällt auf.

Die Voraussetzungen.

Bedingung	**Realistisch?**	
B ist unabhängig	schwierig	**gemeinsame Ahnen**
Übersetzung ist reproduzierbar	heute meist	Zeitstempel entfernen
S ist vollständig prüfbar	teils	Millionen Zeilen
Betriebssystem sauber	offen	dieselbe Frage

Die letzte Zeile verschiebt das Problem nur. Auch das Betriebssystem, der Binder und der Prozessor könnten den Angriff tragen — und für jede dieser Schichten stellt sich die Frage neu. Es gibt keinen Punkt, an dem die Prüfung endet.

Das ist wörtlich die Lage aus Kapitel 4: Um einem System zu trauen, braucht man ein stärkeres, und dessen Vertrauenswürdigkeit ist wieder offen. Praktisch bricht man die Kette irgendwo ab und macht die Annahme explizit — mehr geht nicht.

Reproduzierbare Übersetzung ist die praktische Antwort

Die Forderung	Derselbe Quelltext ergibt bei jedem Bytes für Byte dasselbe Ergebnis
Die Wirkung	Viele Unabhängige können dasselbe Binärprogramm bestätigen
Die Grenze	Ein Angriff im gemeinsamen Werkzeug bleibt unentdeckt

Man ersetzt einen Beweis durch verteiltes Vertrauen. Das ist schwächer als ein Beweis und erheblich besser als nichts — und es ist derselbe Kompromiss wie beim Umgang mit dem Halteproblem: Man löst das Problem nicht, man umgeht es in den Fällen, die vorkommen.

Thompson[26] beschrieb den Angriff in seiner Turing-Award-Rede; Hofstadter[12] entwickelte das Bild des seltsamen Kreises, das hier in technischer Form auftritt; Sipser[24] liefert die theoretische Grundlage über den Rekursionssatz.

8 Das Programm, das sich selbst kopiert

Von Neumanns Frage

1948, vor der Entdeckung der DNS-Struktur, fragte JOHN VON NEUMANN: Wie kann ein Automat einen Automaten seiner eigenen Art bauen?

Das Problem	Eine Bauanleitung für X muss X beschreiben — also größer sein
Der naive Schluss	Selbstreproduktion sei unmöglich
Die Lösung	**Die Anleitung wird zweimal verwendet: ausgeführt und kopiert**

Von Neumanns Aufbau.

Teil	Aufgabe	
A	Baut, was die Anleitung sagt	universeller Konstrukteur
B	**Kopiert die Anleitung**	
C	Steuert beide	
Φ	**Anleitung für** $A + B + C$	nicht für sich selbst

Die letzte Zeile löst das Paradox. Φ beschreibt nur die Maschinerie, nicht sich selbst — deshalb muss sie nicht größer sein als sich selbst. Der Kopiervorgang B vervielfältigt sie, ohne sie zu verstehen.

	Von Neumann 1948	Biologie 1953
Anleitung	Φ	**DNS**
Ausführen	*A* liest Φ	**Translation**
Kopieren	*B* vervielfältigt Φ	**Replikation**
Zwei Rollen	vorhergesagt	bestätigt

Von Neumann sagte die Doppelrolle des Erbmaterials fünf Jahre vor der Aufklärung der DNS-Struktur voraus — allein aus einer logischen Überlegung über Selbstreproduktion. Das ist eine der bemerkenswertesten Vorhersagen der Wissenschaftsgeschichte.

Der Rekursionssatz als Ursache

Die Verbindung	Von Neumanns Aufbau ist der Beweis des Rekursionssatzes in Hardware
Der Kunstgriff	Ein Teil wird gelesen *und* kopiert — wie `%r` im Quine
Die Folge	**Selbstreproduktion ist in jedem universellen System möglich**

Damit ist die Frage entschieden, ob Selbstreproduktion etwas Besonderes ist: Sie ist nichts anderes als der Selbstbezug aus Kapitel 2, und sie steht jedem hinreichend mächtigen System kostenlos zur Verfügung.

Erscheinung	Träger	Anleitung
Quine	Programm	**Zeichenkette im Text**
Computervirus	Programm	sich selbst
Zelle	Molekül-maschinerie	DNS
Zellularautomat	Zellgitter	Muster

Warum es keinen vollständigen Virenschutz geben kann

1	„Ist dieses Programm ein Virus?“ ist eine Verhaltensfrage.
2	Nach Rice ist sie unentscheidbar.
3	Mit dem Rekursionssatz baut man zu jedem Prüfer ein Gegenbeispiel.

Ein Virus, das den Prüfer abfragt und sich nur dann fortpflanzt, wenn der Prüfer es für harmlos hält, ist mit dem Rekursionssatz konstruierbar. Deshalb arbeiten Schutzprogramme mit Signaturen und Heuristiken — also syntaktisch, nicht semantisch.

Aufgabe 8.1.

(a) Rechnen Sie die Größenbilanz eines Selbstkopierers durch.
(b) Untersuchen Sie die Fehlerrate beim Kopieren.
(c) Prüfen Sie, warum Virenschutz nur teilweise gelingt.

Lösung Schritt für Schritt.

(a) Die Größenbilanz.
Der naive Ansatz. Das Programm speichert sich selbst als Text.

Stufe	Code	Gespeicherter Text	Gesamt
1	100	0	100
2	100	100	200
3	100	200	300
n	100	$100(n-1)$	$\to \infty$

Der Text müsste sich selbst enthalten, und das führt in einen unendlichen Regress. Genau dieser Regress ließ von Neumann zunächst vermuten, Selbstreproduktion sei unmöglich.

Der richtige Ansatz.

Teil	Zeichen	Beschreibt	
Code	100	—	wird ausgeführt
Text Φ	100	**den Code**	wird kopiert
Gesamt	**200**		endlich

Φ beschreibt nur den Code, nicht sich selbst — deshalb bleibt es bei hundert Zeichen. Der Code gibt erst Φ aus, dann Φ nochmals in Anführungszeichen. Das ist die Bilanz des Quine aus Kapitel 2: fünfzig Prozent Code, fünfzig Prozent Daten.

Die Bilanz in der Biologie.

Organismus	Basenpaare	Gene	Anteil kodierend
Virus	≈ 5000	≈ 10	$> 90\,\%$
Bakterium	$4 \cdot 10^6$	4000	$\approx 85\,\%$
Hefe	$1{,}2 \cdot 10^7$	6000	$\approx 70\,\%$
Mensch	$3 \cdot 10^9$	$\approx 20\,000$	$\approx 2\,\%$

Die Doppelrolle bleibt in allen Zeilen dieselbe: Die DNS wird gelesen und kopiert. Der Anteil kodierender Abschnitte sinkt drastisch mit der Größe — die Anleitung wächst langsamer als das, was mitgeschleppt wird.

(b) Die Fehlerrate beim Kopieren.
Die Frage. Wie genau muss ein Selbstkopierer sein, damit die Information erhalten bleibt?
Die Fehlerschwelle.

$$\mu \cdot L < 1,$$

mit Fehlerrate μ je Zeichen und Länge L.

System	μ	L	μL
RNS-Virus	10^{-4}	10^{4}	**1,0** (Grenze)
DNS-Virus	10^{-6}	10^{5}	0,1
Bakterium	10^{-9}	$4 \cdot 10^{6}$	0,004
Mensch	10^{-10}	$3 \cdot 10^{9}$	0,3
Datenträger	10^{-15}	10^{13}	0,01

Die erste Zeile liegt genau an der Grenze — RNS-Viren mutieren so schnell, wie es gerade noch geht, ohne ihre Information zu verlieren. Das ist keine Nachlässigkeit, sondern eine Strategie: Sie erkaufen Anpassungsfähigkeit mit einer Fehlerrate am Rand des Erträglichen.

Die Schranke ist informationstheoretisch, nicht biologisch — sie gilt für jeden Selbstkopierer, auch für Programme auf Datenträgern. Länge und Genauigkeit stehen in einem festen Verhältnis, und wer länger werden will, muss genauer kopieren.

Die Konsequenz für die Reihenfolge.

Das Henne-Ei	Ein langes Erbgut braucht gute Kopierenzyme; die stehen im Erbgut
Der Ausweg	Erst kurz und ungenau, dann schrittweise länger und genauer
Die Parallele	**Bootstrapping aus Kapitel 7**

Die Lage ist genau die des Übersetzers: Man braucht das Werkzeug, um das Werkzeug zu bauen. Und die Lösung ist dieselbe — eine Kette von Stufen, deren jede die nächste ermöglicht, beginnend mit etwas sehr Einfachem.

(c) Warum Virenschutz nur teilweise gelingt.
Das Gegenbeispiel. Angenommen, ein Prüfer `istVirus(p)` arbeitet fehlerfrei.

```
def V():
s = <mein eigener Quelltext>
if istVirus(s):
return # harmlos bleiben
else:
kopiere_mich_ueberallhin()
```

`istVirus(V)`	**V tut**	**Ist V ein Virus?**	
`true`	nichts	nein	Prüfer irrt
`false`	**kopiert sich**	ja	Prüfer irrt

Der Prüfer irrt in beiden Fällen. Der Bau von V ist durch den Rekursionssatz gesichert, also existiert das Gegenbeispiel zu jedem *Prüfer. Es ist wörtlich Turings Beweis, angewandt auf die Frage nach Schadsoftware.*

Was Schutzprogramme stattdessen tun.

Verfahren	**Art**	**Grenze**
Signaturen	**syntaktisch**	nur Bekanntes
Heuristiken	syntaktisch	Fehlalarme
Sandkasten	**beschränkte Zeit**	Virus wartet ab
Verhaltens-überwachung	zur Laufzeit	Schaden entsteht
Weiße Listen	**Umkehrung**	wirksam, unbequem

Alle fünf umgehen die Unentscheidbarkeit auf einem der Wege aus Kapitel 5: syntaktisch statt semantisch, beschränkt statt unbeschränkt, oder in eine Richtung sicher statt in beide.

Die letzte Zeile ist die einzige, die im Grundsatz funktioniert: Man erlaubt nur, was ausdrücklich zugelassen ist. Damit stellt sich die unentscheidbare Frage nicht mehr — man hat sie durch eine Verwaltungsentscheidung ersetzt. Wieder der Verzicht auf Allgemeinheit als Preis für eine Garantie.

Der Wettlauf ist strukturell nicht zu gewinnen

Der Prüfer	Muss alle Fälle erkennen
Der Angreifer	Muss einen finden, den der Prüfer verfehlt
Der Satz	**So einen gibt es immer**

Das ist keine Aussage über die Qualität heutiger Schutzprogramme, sondern über die Aufgabe. Sie erklärt, warum es keine endgültige Lösung gibt und warum jede Verbesserung nur die Kosten für den Angreifer erhöht — was in der Praxis genügt und in der Theorie nichts beweist.

Von Neumann[17] entwarf die Theorie selbstreproduzierender Automaten; Dawkins[8] popularisierte den Selbstkopierer als Denkfigur; Hofstadter[12] verbindet Quines, DNS und Gödels Konstruktion zu einem Bild.

9 Das System, das sich selbst beschreibt

Der Katalog im Katalog

Eine Datenbank speichert ihre eigene Struktur in Tabellen — und diese Tabellen stehen im Katalog.

Tabelle	Enthält	Steht selbst in
`kunden`	Kundendaten	`tabellen`
`tabellen`	alle Tabellen	**`tabellen`**
`spalten`	alle Spalten	`tabellen`

Die mittlere Zeile schließt den Kreis: Die Tabelle, die alle Tabellen aufzählt, zählt sich selbst auf. Das ist kein Fehler und kein Kunstgriff, sondern die einzige widerspruchsfreie Möglichkeit — jede Ausnahme wäre eine Sonderbehandlung, die man überall mitschleppen müsste.

Und es funktioniert, weil hier die Umkehrung aus Kapitel 2 fehlt. Der Katalog behauptet nichts über sich, er enthält sich nur. Selbstbezug ohne Negation ist harmlos.

Wie das Henne-Ei aufgelöst wird.

1	Beim Anlegen der Datenbank werden die Katalogtabellen **fest verdrahtet** erzeugt.
2	Danach trägt das System sie in den Katalog ein.
3	Ab jetzt läuft alles über den normalen Weg.

Wieder ein Bootstrapping wie in Kapitel 7: eine anfängliche Handarbeit, danach schließt sich der Kreis und die Sonderbehandlung wird weggeworfen.

Reflexion in Programmiersprachen

Stufe	Kann	Beispiel
Introspektion	**sich befragen**	Welche Methoden hat dieses Objekt?
Intercession	**sich ändern**	Methode zur Laufzeit hinzufügen
Meta-programmierung	**Code erzeugen**	Makros, Vorlagen
Selbst-auswertung	**sich ausführen**	`eval`

Vier Stufen zunehmender Selbstbezüglichkeit. Die erste ist harmlos und nützlich; die letzte macht statische Prüfung unmöglich, denn was ausgeführt wird, steht erst zur Laufzeit fest.

Mit `eval`	Statische Prüfung	
Typprüfung	unmöglich	Typ erst zur Laufzeit
Optimierung	eingeschränkt	
Sicherheitsanalyse	unmöglich	**beliebiger Code**
Toter Code	nicht erkennbar	könnte gerufen werden

Das ist derselbe Handel wie überall in diesem Band: Man gewinnt Ausdruckskraft und verliert Prüfbarkeit. Deshalb verbieten viele Projekte `eval` ausdrücklich — nicht weil es schlecht wäre, sondern weil es alle Werkzeuge blind macht.

Die Metaebenen

Ebene	Gegenstand	Beispiel
0	Daten	Kundenname
1	Schema	Spalte „Name“, Typ Text
2	Metaschema	**„Spalten haben Typen“**
3	Metameta-schema	= Ebene 2

Die letzte Zeile ist die Pointe: Ab Ebene 2 wird der Turm nicht höher, weil das Metaschema sich selbst beschreiben kann. Der unendliche Regress bricht ab, indem der Kreis sich schließt — genau das meint Hofstadters Bild von der Hierarchie, die nach oben wieder unten ankommt.

Aufgabe 9.1.

(a) Fragen Sie einen Katalog nach sich selbst.
(b) Untersuchen Sie, wo der Metaturm abbricht.
(c) Prüfen Sie, was Reflexion an Prüfbarkeit kostet.

Lösung Schritt für Schritt.

(a) Der Katalog über sich selbst.
Die Abfrage.

```
SELECT name FROM tabellen WHERE name = 'tabellen';
```

Ergebnis	Zeilen	
`tabellen`	1	sie enthält sich

Die Zählung in einer typischen Datenbank.

Art	Anzahl	Spalten	
Nutzertabellen	42	380	
Katalogtabellen	≈ 60	≈ 900	mehr als Nutzer
Gesamt	102	1280	

Die Selbstbeschreibung ist in einer kleinen Datenbank umfangreicher als die eigentlichen Daten. Das ist der Preis der Allgemeinheit: Ein System, das beliebige Schemata verwalten kann, braucht ein Schema für Schemata — und das ist nicht klein.

Eine gefährliche Abfrage.

```
DELETE FROM tabellen WHERE name = 'tabellen';
```

Was geschähe	Die Tabelle, die alle Tabellen kennt, wäre unbekannt
Die Folge	Die Datenbank könnte sich nicht mehr öffnen
Die Abhilfe	Katalogtabellen sind schreibgeschützt

Der Selbstbezug wird hier gefährlich, sobald man ihn verändern darf. Deshalb ist der Katalog lesbar und nicht beschreibbar — die Umkehrung, die den Widerspruch erzeugen würde, ist administrativ verboten.

(b) Wo der Metaturm abbricht.
Der Aufbau.

Ebene	Inhalt	Beschrieben von	Neu?
0	„Meier“	Ebene 1	
1	Spalte `name`: Text	Ebene 2	
2	„Spalten haben Namen und Typen“	Ebene 3	
3	„Begriffe haben Eigenschaften“	**Ebene** 3	**nein**

Auf Ebene 3 wird die Beschreibung so allgemein, dass sie auf sich selbst anwendbar ist. Von da an fügt jede weitere Ebene nichts hinzu, und der Turm schließt sich zum Kreis. Praktisch nennt man Ebene 3 das Metametamodell und hört dort auf.

Wo dasselbe noch vorkommt.

Gebiet	Der Kreis	
Grammatiken	**Grammatik für Grammatiken**	beschreibt sich
Typsysteme	Typ der Typen	Vorsicht nötig
Objektsysteme	Klasse der Klassen	Metaklasse
Dateisysteme	Verzeichnis im Verzeichnis	.

Die zweite Zeile ist die einzige gefährliche. „Der Typ aller Typen ist ein Typ“ führt zu einer Antinomie nach Russells Muster — deshalb schichten Beweisassistenten die Typen in ein Universum von Ebenen, statt den Kreis zu schließen.

(c) Was Reflexion kostet.
Ein Programm ohne Reflexion.

Analyse	Ergebnis	
Aufrufgraph	vollständig	
Toter Code	18 % entfernbar	sicher
Typfehler	0	garantiert
Programmgröße	2,4 MB	

Dasselbe Programm mit einem einzigen `eval`.

Analyse	Ergebnis	
Aufrufgraph	unvollständig	**alles erreichbar**
Toter Code	**0** %	nichts sicher entfernbar
Typfehler	nicht prüfbar	
Programmgröße	**7,9 MB**	alles muss bleiben

Ein einziger Aufruf macht das ganze Programm unanalysierbar. Da `eval` beliebigen Code ausführen kann, ist jede Funktion potentiell erreichbar, und nichts darf entfernt werden. Aus zweieinhalb Megabyte werden acht.

Die Abstufung.

Mittel	Prüfbarkeit	
Nichts	voll	
Introspektion	fast voll	lesen ist harmlos
Namensauflösung zur Laufzeit	teilweise	wenn Namen bekannt
`eval` auf Konstanten	teilweise	auflösbar
`eval` auf Eingaben	keine	**Rice**

Die letzte Zeile verweist zurück auf Kapitel 5. Sobald der auszuführende Code von der Eingabe abhängt, ist jede Frage nach dem Verhalten unentscheidbar — nicht schwierig, sondern unentscheidbar.

Die Regel für den Entwurf

Lesende Reflexion	fast immer unbedenklich
Schreibende Reflexion	Vorsicht — ändert das System zur Laufzeit
Auswertende Reflexion	**Verzicht auf jede statische Garantie**

Die drei Zeilen entsprechen genau den drei Zutaten aus Kapitel 2: Bezug allein ist harmlos, Bezug plus Veränderung wird heikel, Bezug plus unbeschränkte Ausführung erzeugt die Grenze. Es ist dieselbe Skala, hier als Entwurfsregel.

Kleene[14] lieferte mit dem Rekursionssatz die Grundlage jeder Selbstbeschreibung; Post[19] untersuchte die Systematik solcher Systeme; Sipser[24] verbindet beides mit der Praxis der Programmanalyse.

Part IV

Der Kreis im Denken

10 Das Gehirn, das sich selbst modelliert

Ein System, das sich selbst untersucht

Band 14 hatte eine Eigentümlichkeit, die keiner der anderen Bände teilt.

Der Gegenstand	Das Gehirn
Das Werkzeug	**Dasselbe Gehirn**
Die Frage	Setzt das eine Grenze?

Die Frage klingt tiefsinnig und ist zunächst zu präzisieren. Ein Modell muss nicht so groß sein wie sein Gegenstand — sonst wäre Physik unmöglich. Es kommt darauf an, was genau modelliert werden soll.

Modelliert werden soll	**Möglich?**	
Die Bauprinzipien	ja	**Band 14**
Statistische Kennwerte	ja	
Ein anderes Gehirn im Detail	im Prinzip	Speichergrenze
Das eigene, vollständig, in sich	**nein**	**Größen-argument**
Das eigene, künftiges Verhalten	**nein**	**Diagonal-argument**

Nur die letzten beiden Zeilen sind grundsätzlich ausgeschlossen, und aus verschiedenen Gründen. Die vierte scheitert am Platz, die fünfte am Selbstbezug — und nur die fünfte ist wirklich interessant.

Die beiden Grenzen

Erstens: das Größenargument.

Die Behauptung	Ein vollständiges Modell eines Systems braucht mindestens so viel Information wie das System
Die Folge	Ein Teil kann nicht das Ganze vollständig enthalten
Die Einschränkung	Gilt nur für *vollständige* Modelle

Die dritte Zeile entwertet das Argument fast völlig. Kein wissenschaftliches Modell ist vollständig, und keines will es sein. Die Newtonsche Mechanik beschreibt das Sonnensystem, ohne es zu enthalten. Das Argument schließt nur eine Vollständigkeit aus, die niemand anstrebt.

Zweitens: das Diagonalargument.

1	Angenommen, ein Verfahren sagt mein künftiges Verhalten voraus.
2	Man teile mir die Vorhersage mit.
3	Ich tue das Gegenteil.
4	**Die Vorhersage war falsch.**

Das ist Turings Beweis in Alltagsform, und es ist ein echtes Ergebnis: Es kann kein Verfahren geben, das ein System zuverlässig vorhersagt und dem System dabei mitgeteilt wird. Bemerkenswert ist die Bedingung in Schritt 2 — ohne Mitteilung fällt das Argument.

Vorhersage	**möglich?**	
Ohne Mitteilung	im Prinzip ja	nur schwer
Mit Mitteilung	**nein**	**Diagonale**
Über andere	ja	
Über sich selbst, intern	nein	

Die Grenze betrifft also nicht die Vorhersagbarkeit an sich, sondern die mitgeteilte *Vorhersage. Das ist eine schwächere und präzisere Aussage als die üblichen Behauptungen über die Unergründlichkeit des Geistes — und sie gilt ebenso für ein Thermostat mit einem Ausgang.*

Das Gödel-Argument gegen Maschinen

LUCAS 1961 und PENROSE 1989 vertraten dasselbe Argument.

1	Sei der Geist eine Maschine M, also ein formales System.
2	Dann gibt es einen Gödelsatz G_M, den M nicht beweisen kann.
3	Ich sehe aber, dass G_M wahr ist.
4	**Also bin ich keine Maschine.**

Einwand 1	Ich sehe G_M nur ein, wenn ich *weiß*, dass M widerspruchsfrei ist
Einwand 2	**Meine eigene Widerspruchsfreiheit kann ich nicht beweisen** — zweiter Gödelsatz
Einwand 3	Ich müsste M kennen — ein vollständiges Modell meiner selbst
Einwand 4	Menschen machen Fehler, sind also nicht offenkundig widerspruchsfrei

Einwand 2 ist der entscheidende. Auch eine Maschine kann sagen: „Wenn ich widerspruchsfrei bin, dann ist G_M wahr" — das beweist sie sogar problemlos. Was sie nicht kann, ist die Voraussetzung zu beweisen. Und der Mensch kann das ebenso wenig.

Das Argument setzt also genau die Fähigkeit voraus, die es beweisen soll. Es ist seit sechzig Jahren in Umlauf und in derselben Zeit widerlegt, und die Widerlegung ist nicht umstritten.

Was daraus nicht folgt

Behauptung	Urteil	
Der Geist ist eine Maschine	unbewiesen	
Der Geist ist keine Maschine	unbewiesen	
Gödel entscheidet die Frage	**falsch**	**beide Richtungen**

Die Widerlegung des Lucas-Penrose-Arguments zeigt nicht, dass der Geist eine Maschine ist. Sie zeigt nur, dass Gödels Sätze zu dieser Frage nichts beitragen. Das ist eine bescheidenere und sicherere Aussage — und sie wird von beiden Lagern ungern gehört.

Aufgabe 10.1.

(a) Rechnen Sie das Größenargument durch.
(b) Führen Sie das Vorhersageparadox aus.
(c) Prüfen Sie das Gödel-Argument im Detail.

Lösung Schritt für Schritt.

(a) Das Größenargument.
Die Zahlen aus Band 14.

Größe	Wert	
Synapsen	$\approx 10^{14}$	
Bit je Synapse	≈ 5	Schätzung
Speicherbedarf	$\approx \mathbf{6 \cdot 10^{13}}$ **Byte**	60 TB

Die Selbstmodellierung.

Anspruch	Nötig	Vorhanden
Vollständig, in sich	60 TB	60 TB (schon belegt)
Auf einem Rechner	60 TB	verfügbar
Bauprinzipien	$\approx 10^5$ Byte	trivial

Die erste Zeile scheitert, weil der Speicher schon durch das Gehirn selbst belegt ist. Die zweite zeigt, dass es keine grundsätzliche Grenze ist — sechzig Terabyte sind heute ein Serverschrank. Die dritte ist das, was Band 14 tatsächlich tut.

Das Größenargument ist also kein tiefes Rätsel, sondern eine Buchhaltungsfrage. Es schließt nur die Selbstmodellierung in sich *aus und sagt nichts über die Wissenschaft vom Gehirn.*

Der Vergleich mit anderen Wissenschaften.

Gegenstand	Zustands-größe	Modellgröße	Verhältnis
Sonnensystem	10^{50}	10^{3}	10^{-47}
Wetter	10^{44}	10^{9}	10^{-35}
Gehirn	10^{14}	10^{5}	10^{-9}

In allen drei Fällen ist das Modell verschwindend klein gegen den Gegenstand, und in allen dreien funktioniert es. Wissenschaft besteht darin, das Wesentliche vom Zustand zu trennen — nicht darin, den Zustand zu speichern.

(b) Das Vorhersageparadox.
Der Aufbau. Ein Gerät sagt voraus, ob ich in einer Minute die linke oder die rechte Hand hebe.

Vorhersage	Mitgeteilt?	Ich tue	Richtig?
links	nein	links	ja
links	**ja**	**rechts**	**nein**
rechts	ja	links	nein

Sobald die Vorhersage mitgeteilt wird, kann ich sie widerlegen. Das setzt nur voraus, dass ich sie verstehe und das Gegenteil tun kann — es braucht keinen freien Willen und keine Besonderheit des Bewusstseins.

Dasselbe an einem Thermostat.

Aufbau	**Verhalten**	
Thermostat allein	vorhersagbar	
Mit Anzeige der Vorhersage und Umkehrregel	**nicht vorhersagbar**	**dieselbe Diagonale**

Ein Bauteil für zwei Euro erzeugt dasselbe Paradox wie ein Gehirn, sobald es die Vorhersage lesen und umkehren kann. Damit ist gezeigt, dass das Argument nichts über das Besondere am Menschen sagt — es sagt etwas über rückgekoppelte Systeme.

Das ist die nüchternste Antwort auf die Frage nach der Grenze der Selbstmodellierung. Sie ist real, sie folgt aus der Diagonale, und sie ist so allgemein, dass sie über den Geist nichts Besonderes aussagt.

(c) Das Gödel-Argument im Detail.
Die Formalisierung. Sei M ein formales System, das angeblich mein mathematisches Denken erfasst.

Behauptet	Ich beweise G_M, M nicht — also $M \neq$ ich
Tatsächlich	Ich beweise nur „$\mathrm{Kons}(M) \to G_M$“
Und	**Das beweist M ebenfalls**

Aussage	**Ich**	M	**Unterschied?**
$\mathrm{Kons}(M) \to G_M$	ja	ja	nein
G_M	nur wenn $\mathrm{Kons}(M)$	nein	fraglich
$\mathrm{Kons}(M)$	**nein**	nein	nein

In keiner Zeile besteht ein Unterschied. Die dritte ist die entscheidende: Ich kann meine eigene Widerspruchsfreiheit nicht beweisen — der zweite Gödelsatz gilt für mich ebenso, sofern ich ein formales System bin. Das Argument setzt voraus, was es zeigen will.

Der übliche Ausweg und sein Preis.

Der Ausweg	„Aber ich *sehe* doch, dass ich widerspruchsfrei bin“
Der Einwand	Menschen glauben regelmäßig Widersprüchliches
Der Beleg	Freges System war widerspruchsfrei geglaubt — Vorwort

Frege hielt sein System zwanzig Jahre lang für widerspruchsfrei, und er war einer der sorgfältigsten Denker seiner Zeit. Die Behauptung, man erkenne die eigene Widerspruchsfreiheit unmittelbar, ist empirisch schwer zu halten.

Was übrig bleibt.

Sicher	Kein System kann sein eigenes künftiges Verhalten mitgeteilt vorhersagen
Sicher	Kein System beweist seine eigene Widerspruchsfreiheit
Offen	Ob der Geist ein formales System ist
Unbrauchbar	Gödel als Argument in dieser Frage

Zwei sichere Ergebnisse, eine offene Frage und ein untaugliches Argument. Das ist eine bescheidene Bilanz für ein Kapitel, das oft als Höhepunkt solcher Bücher auftritt — und sie ist die einzige, die sich vertreten lässt.

Warum die Versuchung so groß ist

Der Reiz	Ein mathematischer Satz scheint eine philosophische Frage zu entscheiden
Der Fehler	Die Voraussetzungen des Satzes werden übergangen
Die Regel	**Wer Gödel außerhalb der Logik anwendet, prüfe zuerst die drei Bedingungen**

Widerspruchsfreiheit, genug Arithmetik, aufzählbare Axiome — Kapitel 4. Fast jede Anwendung des Satzes auf Geist, Gesellschaft oder Erkenntnis scheitert schon an der dritten Bedingung, weil der Gegenstand gar kein formales System mit aufzählbaren Axiomen ist.

Hofstadter[12] entwickelte das Bild des seltsamen Kreises für das Selbstmodell und führte es später weiter[13]; Penrose[18] vertrat die bekannteste Fassung des Gödel-Arguments gegen Maschinen.

11 Die Grenze anderer Bauart

Eine Grenze ohne Selbstbezug

Band 15 endete mit einem Bauteil, dessen Zustand sich grundsätzlich nicht vollständig auslesen lässt. Das ist ebenfalls eine Grenze — und sie entsteht anders.

	Halteproblem	**Auslesegrenze**
Ursache	**Selbstbezug**	**Komplementarität**
Beweisart	Diagonale	Rechnung
Betrifft	**alle universellen Systeme**	**die Physik**
Umgehbar durch	Sprache schwächen	gar nicht

Die letzte Zeile ist der wichtigste Unterschied. Die Grenzen der Kapitel 4 bis 6 lassen sich durch Verzicht auf Mächtigkeit vermeiden — man baut eine schwächere Sprache und gewinnt Entscheidbarkeit. Bei der Quantenmechanik gibt es diesen Handel nicht.

Die Auslesegrenze im Einzelnen.

Ein Qubit trägt	Zwei reelle Zahlen
Eine Messung liefert	**Ein bit**
Der Rest	Ist nach der Messung fort, nicht bloß unbekannt

Das ist die Holevo-Schranke aus Band 15, Kapitel 9. Sie ist bewiesen, scharf und hat mit Selbstbezug nichts zu tun — sie folgt aus der Struktur des Zustandsraums.

Drei Arten von Grenzen

Art	Ursache	Beispiel	Band
Logisch	**Selbstbezug**	Halteproblem	2
Informations-theoretisch	**Abzählung**	Kolmogorow-Komplexität	7
Physikalisch	**Naturgesetz**	Auslesegrenze	15

Die mittlere Zeile ist eine eigene Kategorie und wird oft mit der ersten verwechselt. Dass fast alle Zeichenketten unkomprimierbar sind, folgt aus einem Abzählargument: Es gibt weniger kurze Beschreibungen als lange Objekte. Kein Selbstbezug nötig.

Wo sie ineinandergreifen.

Chaitins Satz	Ein System kann nur endlich viele Aussagen der Form „$K(x) > n$“ beweisen
Die Folge	**Unvollständigkeit aus Informationsmangel**
Die Pointe	Ein zweiter, unabhängiger Beweis von Gödels Satz

Chaitin bewies die Unvollständigkeit nicht über die Diagonale, sondern über einen Größenvergleich: Ein System mit n bit Axiomen kann keine Zeichenkette als komplexer denn n bit nachweisen. Damit hat dieselbe Grenze zwei ganz verschiedene Ursachen.

Für diesen Band ist das eine wichtige Einschränkung. Selbstbezug erklärt viele Grenzen — er erklärt sie nicht alle, und manchmal ist er nicht einmal die tiefste Erklärung.

Aufgabe 11.1.

(a) Vergleichen Sie die Beweise beider Grenzen.
(b) Rechnen Sie Chaitins Argument durch.
(c) Prüfen Sie, ob die Quantenmechanik die Berechenbarkeit ändert.

Lösung Schritt für Schritt.

(a) Zwei Beweise im Vergleich.

Das Halteproblem.

1	Annahme: H existiert.
2	Baue D mit Selbstbezug und Umkehrung.
3	D auf D ergibt Widerspruch.

Das Klonverbot.

1	Annahme: U mit $U\|\psi\rangle\|0\rangle = \|\psi\rangle\|\psi\rangle$ existiert.
2	Wende auf zwei Zustände an, bilde das Skalarprodukt.
3	Es folgt $z = z^2$, also $z \in \{0, 1\}$ — Widerspruch.

	Halteproblem	**Klonverbot**
Kunstgriff	**Selbst-anwendung**	**Linearität**
Widerspruch aus	Umkehrung	Algebra
Länge	4 Zeilen	4 Zeilen
Braucht Aufzählung	ja	nein

Beide Beweise sind vier Zeilen lang und beide widerlegen eine Annahme. Der Unterschied steht in der letzten Zeile: Das Klonverbot braucht keine Aufzählung, keinen Index, keine Diagonale. Es ist eine Rechnung über zwei Vektoren.

(b) Chaitins Argument.
Der Begriff. $K(x)$ sei die Länge des kürzesten Programms, das x ausgibt.
Die Abzählung.

Länge n	Ketten 2^n	Programme $< n$	Anteil
10	1024	1023	99,9 %
20	$1{,}05 \cdot 10^6$	$\approx 10^6$	99,99 %
n	2^n	$2^n - 1$	

Anteil mit $K(x) \geq n - 10$: mindestens $1 - 2^{-10} = 99{,}9\,\%$

Es gibt weniger kurze Programme als lange Zeichenketten, also sind fast alle Ketten unkomprimierbar. Das ist ein Abzählargument von zwei Zeilen und ohne jeden Selbstbezug.

Der Schluss auf die Unvollständigkeit.

1	Sei F ein System mit einer Beschreibung von c bit.
2	Angenommen, F beweist „$K(x) > n$“ für ein n weit über c.
3	Dann baue: „Durchsuche alle Beweise, bis einer ‚$K(x) > n$‘ zeigt; gib dieses x aus.“
4	Dieses Programm hat etwa $c + \log n$ bit und gibt x aus.
5	**Also** $K(x) \leq c + \log n < n$ — Widerspruch.

System	c (bit)	Grenze für Beweise
Peano-Arithmetik	$\approx 10^4$	$K(x) > 10^4$ unbeweisbar
Mengenlehre	$\approx 10^4$	ebenso

Ein System kann die Komplexität einer Zeichenkette nur bis zu seiner eigenen Beschreibungslänge nachweisen. Da fast alle Ketten weit darüber liegen, sind fast alle wahren Komplexitätsaussagen unbeweisbar — und man kann keine einzige davon angeben.

Das ist eine bemerkenswerte Verschärfung von Gödel. Bei Gödel gibt es einen *unbeweisbaren Satz, den man hinschreiben kann. Bei Chaitin sind es fast alle, und man kann keinen benennen.*

(c) Ändert die Quantenmechanik die Berechenbarkeit?

Frage	Antwort	
Neue berechenbare Funktionen?	**nein**	**Band 15, Kap. 8**
Halteproblem lösbar?	nein	
Schneller?	manchmal	Shor
Neue Grenzen?	**ja**	**Auslesen, Klonen**

Die Quantenmechanik verschiebt die Grenze zwischen praktikabel und unpraktikabel und lässt die zwischen berechenbar und unberechenbar unberührt. Sie fügt dafür Grenzen hinzu, die es klassisch nicht gab — Kopieren und Auslesen.

Die Bilanz für den seltsamen Kreis.

Grenze	Selbstbezug?	
Halteproblem	ja	
Unvollständigkeit	ja	oder Abzählung
Rice	ja	
Hierarchiesätze	ja	
P gegen NP	**versagt**	Kapitel 6
Kolmogorow	nein	Abzählung
Klonverbot	nein	Linearität
Holevo	nein	Dimension

Vier von acht. Der seltsame Kreis erklärt die Grenzen der Berechenbarkeit und einen Teil der Logik — und er erklärt weder die Effizienz noch die Physik. Das ist die ehrliche Reichweite der These dieses Bandes.

Eine Denkfigur ist kein Naturgesetz

Die Versuchung	Alle Grenzen als Erscheinungsformen einer Idee zu deuten
Der Fehler	Verschiedene Ursachen werden eingeebnet
Die Probe	**Nennen Sie den Beweis** — nicht die Analogie

Wer das Klonverbot als „Selbstbezug in der Physik“ bezeichnet, hat eine hübsche Formulierung und keinen Beweis. Der Beweis ist eine Zeile Algebra über Skalarprodukte, und dort kommt kein Selbstbezug vor. Die Ähnlichkeit liegt im Ergebnis, nicht im Grund.

Diese Unterscheidung ist der Grund, warum dieses Kapitel im Buch steht. Ein Band über eine Denkfigur muss zeigen, wo sie nicht greift — sonst wird aus einer Einsicht eine Weltanschauung.

Aaronson[1] vergleicht die Grenzen der Logik, der Komplexität und der Physik systematisch; Zurek[28] erklärt, warum die Auslesegrenze physikalisch unvermeidlich ist; Chaitin[4] bewies die Unvollständigkeit auf einem zweiten, unabhängigen Weg.

12 Was die Reihe gezeigt hat

Sechzehn Bände, eine Landkarte

Bd.	Thema	Die Grenze
0	Logik	keine — entscheidbar
1	Algorithmen	
2	Berechenbarkeit	**Halteproblem**
3	Komplexität	**P gegen NP offen**
4	Architektur	Universalität
5	Betriebssysteme	Verklemmung
6	Übersetzerbau	**Bootstrapping**
7	Information	**Kolmogorow**
8	Kryptographie	unbewiesene Annahmen
9	Verteilte Systeme	**FLP-Unmöglichkeit**
10	Datenbanken	**Katalog im Katalog**
11	Graphen	Nullmodell
12	Symbolische KI	Unentscheidbarkeit
13	Lernen	**Rauschgrenze**
14	Gehirn	**Selbstmodellierung**
15	Quanteninformatik	**Auslesegrenze**

Jeder Band endete mit einer Grenze, und keine davon war ein Mangel der Technik. Sie folgten alle aus dem, was das jeweilige System kann — und die meisten aus derselben Bewegung.

Nach Ursache geordnet.

Ursache	Bände	Anteil
Selbstbezug	2, 6, 10, 12, 14	5
Abzählung	7, 13	2
Physik	15	1
Offen	3, 8	2
Andere	5, 9, 11	3

Fünf von dreizehn — keine Mehrheit, aber die größte Einzelgruppe. Und es sind die grundsätzlichsten: Wo Selbstbezug die Ursache ist, hilft kein Fortschritt, sondern nur Verzicht auf Mächtigkeit.

Die drei Lehren

Erstens: Grenze und Mächtigkeit sind dasselbe.

System	Mächtig genug für	Daher unentscheidbar
Presburger	nur Addition	nein
Peano	**auch Multiplikation**	ja
Reguläre Ausdrücke	kein Speicher	nein
Turing-maschinen	**unbegrenzter Speicher**	ja

Die Schwelle liegt jedes Mal genau dort, wo das System anfängt, über sich selbst sprechen zu können. Das ist kein Zufall und keine Metapher, sondern die Voraussetzung des Beweises.

Zweitens: Der Verzicht ist eine Entwurfsentscheidung.

Man will	Man verzichtet auf	Beispiel
Terminierung	unbeschränkte Rekursion	Beweisassistent
Auswertbarkeit	volle Rekursion	Abfragesprache
Prüfbarkeit	`eval`	Projektregel
Sicherheit	**Ausführungs-rechte**	weiße Liste

Das ist die praktisch nützlichste Einsicht dieses Bandes. Wer eine Garantie braucht, muss Mächtigkeit aufgeben — und die Frage ist nicht, ob, sondern welche.

Drittens: Unentscheidbar ist eine Aussage über den schlimmsten Fall.

Aufgabe	Theorie	Praxis
Terminierung prüfen	unentscheidbar	$\approx 87\,\%$ gelöst
Typfehler finden	unentscheidbar	Standard
Erfüllbarkeit	NP-vollständig	Millionen Variablen
Viren erkennen	unentscheidbar	teilweise

Die schwierigen Fälle existieren und sind Diagonalkonstruktionen — Programme, die eigens gebaut wurden, um einen Prüfer zu täuschen. Menschen schreiben solche Programme nicht, außer als Gegenbeispiel. Deshalb funktioniert die Praxis trotz der Theorie.

Aufgabe 12.1.

(a) Ordnen Sie eine neue Grenze richtig ein.
(b) Prüfen Sie eine Gödel-Anwendung auf ihre Voraussetzungen.
(c) Bestimmen Sie, was von der These bleibt.

Lösung Schritt für Schritt.

(a) Eine neue Grenze einordnen.

Der Fall. Der Satz von Fischer, Lynch und Paterson aus Band 9: In einem asynchronen System mit einem möglichen Ausfall gibt es kein sicheres Einigungsverfahren.

Prüffrage	**Antwort**
Selbstbezug im Beweis?	nein
Diagonale?	nein
Abzählung?	nein
Was dann?	**Ein Fortsetzungsargument über Ausführungen**

Der Beweis zeigt, dass es stets eine unentschiedene Ausführung gibt, die sich verlängern lässt. Kein Selbstbezug, kein Index, keine Umkehrung — eine ganz andere Bauart. Die Grenze ist echt und gehört nicht zu diesem Band.

Bauart	**Kennzeichen**	**Beispiele**
Diagonal	Index, Umkehrung	Halteproblem, Gödel
Abzählend	**zu wenige kurze Objekte**	Kolmogorow
Algebraisch	Rechnung über Struktur	**Klonverbot**
Fortsetzend	**immer ein Schritt weiter**	FLP
Zählend	Schubfach	untere Schranken

Fünf Bauarten von Unmöglichkeitsbeweisen. Wer einer neuen Grenze begegnet, prüfe zuerst, welche vorliegt — davon hängt ab, ob sich durch Verzicht auf Mächtigkeit etwas gewinnen lässt.

(b) Eine Gödel-Anwendung prüfen.
Die Behauptung. „Nach Gödel kann keine Gesellschaft ihre eigenen Regeln vollständig festlegen.“

Voraussetzung	Erfüllt?	
Formales System	nein	**Regeln sind vage**
Enthält Arithmetik	nein	
Aufzählbare Axiome	nein	**ändern sich**
Widerspruchsfrei	nein	offenkundig nicht

Keine der vier Voraussetzungen ist erfüllt. Der Satz sagt über Gesellschaften also gar nichts — weder dass sie vollständig sein können noch dass sie es nicht können. Die Behauptung benutzt einen mathematischen Namen für eine Vermutung.

Eine Anwendung, die trägt.

Voraussetzung	Erfüllt?	
Formales System	ja	**Beweis-assistent**
Enthält Arithmetik	ja	
Aufzählbare Axiome	ja	
Widerspruchsfrei	angenommen	
Folgerung	**Kann eigene Konsistenz nicht beweisen**	

Hier greift der Satz, und die Folgerung ist konkret: Wer einem Beweisassistenten traut, trifft eine Annahme, die das Werkzeug selbst nicht rechtfertigen kann. Das ist dieselbe Lage wie bei Thompsons Übersetzer in Kapitel 7.

Die Prüfliste.

1	Ist der Gegenstand ein formales System mit festen Regeln?
2	Sind seine Axiome aufzählbar?
3	Enthält es genug Arithmetik zur Selbstkodierung?
4	Wird Widerspruchsfreiheit angenommen oder behauptet?
5	Wird zwischen „wahr“ und „beweisbar“ unterschieden?

Fast jede außermathematische Anwendung scheitert an Frage 1 oder 2. Frage 5 ist die häufigste stillschweigende Verwechslung — und die einzige, die auch innerhalb der Mathematik regelmäßig vorkommt.

(c) Was von der These bleibt.
Die These aus dem Vorwort.

Teilbehauptung	Status	
Universalität erlaubt Selbstbezug	**bewiesen**	Rekursionssatz
Selbstbezug erlaubt die Diagonale	**bewiesen**	
Die Diagonale erzeugt Grenzen	**bewiesen**	Kap. 4–5
Alle Grenzen kommen daher	**falsch**	Kap. 11
Die Diagonale erklärt Effizienz	**falsch**	Kap. 6

Drei von fünf halten stand, und es sind die ersten drei — also die Kette selbst. Was nicht hält, sind die Verallgemeinerungen: dass sie alles erklärt und dass sie über die Berechenbarkeit hinausreicht.

Die Formulierung, die trägt.

Zu stark	„Jede Grenze folgt aus Selbstbezug“
Zu schwach	„Selbstbezug kommt oft vor“
Richtig	**Jedes System, das über sich selbst sprechen kann, hat Fragen über sich, die es nicht beantworten kann**

Die dritte Zeile ist beweisbar, präzise und nicht trivial. Sie sagt nichts über Grenzen, die anderswoher kommen, und sie behauptet keine Unergründlichkeit — nur eine Unvollständigkeit des Selbstwissens.

Ein Schlusswort zur Reihe

Band 0	Ein Schalter, der zwei Zustände unterscheidet
Band 15	Ein Bauteil, dessen Zustand sich nicht vollständig auslesen lässt
Dazwischen	**Sechzehn Bände, in denen jedes Mal dieselbe Frage auftaucht**

Die Frage lautet: Was kann ein System über sich selbst wissen? Sie hat in jedem Band eine andere Gestalt — als Halteproblem, als Bootstrapping, als Katalog, als Selbstmodell, als Messung. Und sie hat nie eine vollständig befriedigende Antwort.

Das ist kein Grund zur Resignation. Die Grenzen sind bekannt, präzise und in ihrer Reichweite verstanden — und das ist mehr, als man von den meisten Wissenschaften sagen kann. Man weiß hier nicht nur, was man kann, sondern auch, was man grundsätzlich nicht kann und warum.

Ein System, das mächtig genug ist, über sich selbst zu sprechen,
hat Fragen über sich, die es nicht beantworten kann.
Man kann diese Grenze aufheben, indem man auf die Mächtigkeit verzichtet.
Etwas anderes kann man nicht tun.

— die These dieses Bandes, auf das reduziert, was bewiesen ist.
Alles Weitere ist Analogie, und Analogien
sind keine Beweise — Kapitel 11.

Franzén[9] räumt mit den verbreiteten Fehlanwendungen auf; Lucas[15] lieferte die bekannteste davon; Hofstadter[13] verteidigt den seltsamen Kreis als Denkfigur, ohne ihn zum Beweis zu erklären.

Part V

Anhang

A Die Bewegung im Überblick

Das Verfahren in seinen Gestalten

	Aufgezählt	Das neue Objekt	Folge
Cantor	alle Folgen	umgekehrte Diagonale	über-abzählbar
Russell	—	$\{x : x \notin x\}$	Wider-spruch
Gödel	alle Beweise	„unbeweisbar“	unvoll-ständig
Turing	alle Programme	„tu das Gegenteil“	unent-scheidbar
Rice	alle Prüfer	Gegenbeispiel	unent-scheidbar
Hartmanis	alle $\mathcal{O}(f)$-Maschinen	simuliert und kehrt um	Hierarchie
Baker u. a.	alle Orakel	zwei Orakel	**Sperre**

Die letzte Zeile ist die Pointe der Reihe: Die Diagonale wird auf sich selbst angewandt und zeigt ihre eigene Grenze. Ein seltsamer Kreis im Beweisverfahren selbst.

Die drei Zutaten und die vier Auswege

Zutat	Ohne sie	Wer verzichtet
Selbstbezug	harmlos	Typentheorie
Umkehrung	harmlos	**Datenbankkatalog**
Zweiwertigkeit	harmlos	dreiwertige Logik
Alle drei	**Widerspruch**	

Gödels Weg	kein Verzicht	**„unbeweisbar“ statt „falsch“**
Der Preis	Unvollständigkeit statt Widerspruch	

Fünf Bauarten von Unmöglichkeitsbeweisen

Bauart	Kennzeichen	Beispiel
Diagonal	Index und Umkehrung	Halteproblem
Abzählend	zu wenige kurze Objekte	Kolmogorow
Algebraisch	Rechnung über Struktur	Klonverbot
Fortsetzend	stets ein Schritt weiter	FLP
Zählend	Schubfachprinzip	untere Schranken

Nur bei der ersten hilft es, die Mächtigkeit des Systems zu verringern. Bei den übrigen liegt die Ursache anderswo, und der Verzicht bringt nichts.

B Landkarte

Wo was steht

Kap.	Thema	Verweise
1	Die Diagonale	Band 2
2	Der Selbstbezug	Band 1, Band 6
3	Universalität	**Band 2, Band 4**
4	Gödel	Band 1
5	Turing	**Band 2**
6	Wo sie versagt	**Band 3**
7	Der Übersetzer	**Band 6, Band 8**
8	Die Selbstkopie	Band 7, Band 14
9	Selbstbeschreibung	**Band 10, Band 12**
10	Das Gehirn	**Band 14**
11	Die Messung	**Band 7, Band 15**
12	Bilanz	alle

Die Reihe im Überblick

Bd.	Titel	Gegenstand
0	Die Architektur der Gewissheit	Logik
1	Die Anatomie der Geschwindigkeit	Algorithmen
2	Die Grenze der Maschine	Berechenbarkeit
3	Der Preis der Berechnung	Komplexität
4	—	Architektur
5	Der Wettlauf der Prozesse	Betriebssysteme
6	Die Verwandlung des Textes	Übersetzerbau
7	Das Maß der Nachricht	Information
8	Die Mathematik des Misstrauens	Kryptographie
9	Die Einigung der Getrennten	Verteilte Systeme
10	Das Gedächtnis der Systeme	Datenbanken
11	Das Muster der Verbindungen	Graphen
12	Der Raum der Möglichkeiten	Symbolische KI
13	Der Schluss aus dem Beispiel	Maschinelles Lernen
14	Das Rechnen des Gehirns	Neurowissenschaft
15	Die Auslöschung der Irrwege	Quanteninformatik
B	**Der seltsame Kreis**	**Selbstbezug**

Sechs Verbindungen zur Mathematikreihe

Diagonalverfahren	Mächtigkeit, Kardinalzahlen
Russells Antinomie	Axiomatische Mengenlehre
Gödel-nummerierung	Eindeutigkeit der Primfaktorzerlegung
Rekursionssatz	Fixpunktsätze
Hierarchiesätze	Wachstumsordnungen
Kolmogorow-Komplexität	Abzählargumente, Schubfachprinzip

Die vierte Zeile ist die tiefste. Der Rekursionssatz ist ein Fixpunktsatz: Er behauptet die Existenz eines Programms, das sich selbst als Argument erhält. Dieselbe Struktur trägt den Banachschen Fixpunktsatz und Gödels Konstruktion.

C Glossar

Begriffe

Aufzählbar	Es gibt ein Programm, das alle Elemente nacheinander ausgibt
Bootstrapping	Ein Werkzeug erzeugt sich über Zwischenstufen selbst
Diagonalverfahren	Ein Objekt bilden, das sich vom n-ten an Stelle n unterscheidet
Entscheidbar	Ein Programm beantwortet die Frage stets richtig und hält
Gödelnummer	Kodierung einer Formel als natürliche Zahl
Halteproblem	Hält Programm p auf Eingabe e?
Kolmogorow-Komplexität	Länge des kürzesten erzeugenden Programms
Metaebene	Beschreibungsebene über einer anderen
Quine	Programm, das seinen eigenen Text ausgibt
Reduktion	Eine Frage mechanisch auf eine andere zurückführen
Reflexion	Ein Programm befragt oder ändert sich zur Laufzeit
Rekursionssatz	Jedes Programm darf seinen eigenen Text benutzen
Relativierung	Ein Beweis gilt auch mit beliebigem Orakel
Satz von Rice	Jede nichttriviale Verhaltenseigenschaft ist unentscheidbar
Seltsamer Kreis	Hierarchie, die aufwärts wieder unten ankommt
Unvollständigkeit	Es gibt wahre, im System unbeweisbare Sätze
Universalität	Ein System kann jedes andere nachahmen

D Zeittafel

Von 1891 bis 2007

1891	**Cantor: das Diagonalverfahren**
1902	Russell: der Brief an Frege
1910	Whitehead, Russell: Typentheorie
1928	Hilbert: das Entscheidungsproblem
1931	**Gödel: Unvollständigkeit**
1934	Gödel, Herbrand: rekursive Funktionen
1935	Kleene: der Rekursionssatz
1936	**Turing: Halteproblem und universelle Maschine**
1936	Church: das Lambda-Kalkül
1943	Post: Postsysteme
1944	Post: Grade der Unlösbarkeit
1946	von Neumann: selbstreproduzierende Automaten
1952	Kleene: *Introduction to Metamathematics*
1953	**Rice: alle nichttrivialen Eigenschaften**
1958	Nagel, Newman: *Gödel's Proof*
1961	Lucas: Gödel gegen die Maschine

1965	**Hartmanis, Stearns: Zeithierarchie**
1965	Davis (Hrsg.): *The Undecidable*
1966	Quine: *The Ways of Paradox*
1967	Rogers: Rekursionstheorie
1970	Conway: das Spiel des Lebens
1971	Cook: NP-Vollständigkeit
1974	Chaitin: Unvollständigkeit aus Informationsmangel
1975	**Baker, Gill, Solovay: Relativierung**
1976	Dawkins: der Selbstkopierer als Bild
1979	**Hofstadter: Gödel, Escher, Bach**
1984	**Thompson: Reflections on Trusting Trust**
1989	Penrose: *The Emperor's New Mind*
1994	Razborov, Rudich: natürliche Beweise
2005	Franzén: die Aufräumarbeit
2007	Hofstadter: *I Am a Strange Loop*
2008	Aaronson, Wigderson: Algebrisierung

Zwei Zeilen verdienen eine Anmerkung.
1935, der Rekursionssatz. Kleene bewies, dass jedes Programm seinen eigenen Text benutzen darf — und machte damit den Selbstbezug von einem Kunstgriff zu einem Werkzeug. Fast jeder Beweis dieses Bandes beruht darauf, meist stillschweigend.
1975, die Relativierung. Die Diagonale wird auf sich selbst angewandt und zeigt ihre eigene Grenze. Nach diesem Ergebnis wusste man, dass die wichtigste offene Frage der Informatik mit der Methode, die alle anderen Grenzen erklärt hatte, nicht zu beantworten ist.

E Kommentierte Quellen

[1] Aaronson, S.: *Why Philosophers Should Care About Computational Complexity*. In: Computability, MIT Press 2013.

Vergleicht die Grenzen der Logik, der Komplexität und der Physik und zeigt, warum sie verschiedene Dinge sind. Der beste vorhandene Text gegen die Einebnung aller Grenzen zu einer.

[2] Baker, T., Gill, J., Solovay, R.: *Relativizations of the P =? NP Question*. SIAM J. Comput. 4(4), 1975.

Ein Beweis darüber, welche Beweise unmöglich sind. *Zeigt mit einer Diagonalkonstruktion, dass die Diagonalkonstruktion für P gegen NP nicht ausreicht — die eleganteste Selbstanwendung des ganzen Gebiets.*

[3] Cantor, G.: *Über eine elementare Frage der Mannigfaltigkeitslehre*. Jahresbericht DMV 1, 1891.

Der Ursprung. *Zwei Seiten, ein Verfahren, und es trägt seither die halbe theoretische Informatik. Cantor sah bereits, dass es allgemeiner ist als sein erster Anlass.*

[4] Chaitin, G.: *Information-Theoretic Limitations of Formal Systems*. J. ACM 21(3), 1974.

Beweist Gödels Satz ein zweites Mal, ohne Selbstbezug: Ein System mit n bit Axiomen kann keine höhere Komplexität nachweisen. Wichtig für diesen Band, weil es zeigt, dass dieselbe Grenze verschiedene Ursachen haben kann.

[5] Church, A.: *An Unsolvable Problem of Elementary Number Theory*. Amer. J. Math. 58, 1936.

Kam von der Seite des Lambda-Kalküls zum selben Ergebnis wie Turing — wenige Monate früher und erheblich schwerer zu lesen. Turings Maschinenbild setzte sich durch, weil man es sich vorstellen kann.

[6] Cook, S.: *The Complexity of Theorem-Proving Procedures*. STOC 1971.

Führt die NP-Vollständigkeit ein. Für diesen Band der Bezugspunkt: Es ist die Frage, an der die Diagonale versagt.

[7] Davis, M. (Hrsg.): *The Undecidable*. Raven Press 1965.

Versammelt die Originalarbeiten von Gödel, Church, Turing, Kleene und Post in einem Band. Wer die Quellen selbst lesen will, findet sie hier — und stellt fest, wie klar sie geschrieben sind.

[8] Dawkins, R.: *The Selfish Gene*. Oxford University Press 1976.

Machte den Selbstkopierer zur allgemeinen Denkfigur. Für diesen Band nützlich als Brücke zwischen von Neumanns Automaten und der Biologie, mit der gebotenen Vorsicht gegenüber der Übertragung.

[9] Franzén, T.: *Gödel's Theorem: An Incomplete Guide to Its Use and Abuse*. A K Peters 2005.

Die Aufräumarbeit. *Geht die verbreiteten Fehlanwendungen einzeln durch und zeigt, an welcher Voraussetzung jede scheitert. Sachlich, geduldig und unbedingt zu empfehlen, bevor man Gödel zitiert.*

[10] Gödel, K.: *Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I*. Monatshefte für Mathematik und Physik 38, 1931.

Die Arbeit selbst. *Etwa fünfundzwanzig Seiten, sehr dicht, und die eigentliche Erfindung — die Arithmetisierung der Syntax — steht in den ersten Abschnitten.*

[11] Hartmanis, J., Stearns, R.: *On the Computational Complexity of Algorithms*. Trans. AMS 117, 1965.

Begründet die Komplexitätstheorie und beweist die Hierarchiesätze — mit derselben Diagonale, nun unter einer Zeitschranke. Der Aufsatz, der aus Berechenbarkeit Effizienz machte.

[12] Hofstadter, D.: *Gödel, Escher, Bach: An Eternal Golden Braid*. Basic Books 1979.

Das Buch, das diesen Band angeregt hat. *Verbindet Gödels Beweis, Quines, DNS, Bachs Kanons und Eschers Bilder zu einer Denkfigur. Nicht immer streng, stets anregend, und nach fünfundvierzig Jahren unübertroffen.*

[13] Hofstadter, D.: *I Am a Strange Loop*. Basic Books 2007.

Die Fortsetzung, konzentriert auf das Selbstmodell. Persönlicher und spekulativer als der Vorgänger; Hofstadter ist dabei ehrlich, dass er eine Denkfigur vorschlägt und keinen Beweis führt.

[14] Kleene, S.: *Introduction to Metamathematics*. North-Holland 1952.

Das Standardwerk der Rekursionstheorie und die Quelle für den Rekursionssatz. Trocken, vollständig und für den technischen Kern dieses Bandes unentbehrlich.

[15] Lucas, J. R.: *Minds, Machines and Gödel*. Philosophy 36, 1961.

Die erste Fassung des Gödel-Arguments gegen den Maschinencharakter des Geistes. Lesenswert als Beispiel für einen sorgfältig vorgetragenen und dennoch falschen Schluss — und für die Hartnäckigkeit, mit der er sich hält.

[16] Nagel, E., Newman, J.: *Gödel's Proof*. New York University Press 1958.

Erklärt den Beweis auf hundert Seiten ohne Formalismus und ohne Verfälschung. Wenn Sie eine Darstellung für Nichtfachleute suchen, diese.

[17] von Neumann, J.: *Theory of Self-Reproducing Automata*. Hrsg. A. Burks, University of Illinois Press 1966.

Die Doppelrolle der Anleitung, fünf Jahre vor der DNS-Struktur. *Aus einer rein logischen Überlegung folgte eine biologische Vorhersage — eine der bemerkenswertesten der Wissenschaftsgeschichte.*

[18] Penrose, R.: *The Emperor's New Mind*. Oxford University Press 1989.

Die bekannteste moderne Fassung des Gödel-Arguments. Glänzend geschrieben, in der Physik lehrreich und im logischen Kernargument nicht haltbar — Kapitel 10 führt aus, warum.

[19] Post, E.: *Recursively Enumerable Sets of Positive Integers and Their Decision Problems*. Bull. AMS 50, 1944.

Ordnet unlösbare Probleme nach Schwierigkeitsgraden und fragt, ob es Grade zwischen entscheidbar und dem Halteproblem gibt. Die Antwort dauerte sechzehn Jahre und lautet ja.

[20] Quine, W. V. O.: *The Ways of Paradox*. Random House 1966.

Enthält den Kunstgriff, mit dem ein Satz ohne Zeigewort auf sich verweist. Nach ihm sind die selbstausgebenden Programme benannt — Quine selbst hat nie eines geschrieben.

[21] Rice, H. G.: *Classes of Recursively Enumerable Sets and Their Decision Problems*. Trans. AMS 74, 1953.

Erheblich stärker als das Halteproblem und viel zu wenig bekannt. *Jede nichttriviale Verhaltenseigenschaft ist unentscheidbar — das begrenzt jedes Werkzeug der Programmanalyse.*

[22] Rogers, H.: *Theory of Recursive Functions and Effective Computability*. McGraw-Hill 1967.

Das umfassende Werk zur Rekursionstheorie. Enthält die Diagonalverfahren in ihrer allgemeinsten Form und ist die Quelle, wenn man die technischen Details braucht.

[23] Russell, B.: *Brief an Frege*, 1902. In: van Heijenoort (Hrsg.), *From Frege to Gödel*, Harvard University Press 1967.

Zwei Absätze, die ein Lebenswerk erschütterten. Freges Antwort und sein gedruckter Nachtrag stehen daneben und sind ein Musterbeispiel wissenschaftlicher Redlichkeit.

[24] Sipser, M.: *Introduction to the Theory of Computation*. 3. Aufl., Cengage 2012.

Das beste Lehrbuch für Berechenbarkeit und Komplexität. Klar, mit sorgfältigen Beweisen und guten Aufgaben — der natürliche Anschluss an Band 2 und 3 der Reihe.

[25] Smullyan, R.: *Gödel's Incompleteness Theorems*. Oxford University Press 1992.

Die technisch saubere Darstellung mit allen Voraussetzungen. Smullyan schrieb daneben Rätselbücher über Selbstbezug, die dasselbe Material auf ganz andere Weise zugänglich machen.

[26] Thompson, K.: *Reflections on Trusting Trust*. Commun. ACM 27(8), 1984.

Drei Seiten, die die Softwaresicherheit verändert haben. *Ein Übersetzer, der eine Hintertür in jeden neuen Übersetzer schreibt, ohne dass sie im Quelltext steht. Der Selbstbezug als Waffe.*

[27] Turing, A.: *On Computable Numbers, with an Application to the Entscheidungsproblem*. Proc. London Math. Soc. 42, 1936.

Die Gründungsurkunde der Informatik. *Enthält das Maschinenmodell, die universelle Maschine und die Unentscheidbarkeit — drei grundlegende Ergebnisse in einem Aufsatz eines Vierundzwanzigjährigen.*

[28] Yanofsky, N.: *The Outer Limits of Reason*. MIT Press 2013.

Ordnet die Grenzen der Sprache, der Mathematik, der Berechnung und der Physik nebeneinander. Populär gehalten und in der Unterscheidung der Ursachen sorgfältiger als die meisten Bücher dieser Art.

Wenn Sie nur drei Bücher kaufen

[12]	**Hofstadter**	Für die Denkfigur und die Freude daran
[24]	**Sipser**	Für die Beweise, sauber und vollständig
[9]	**Franzén**	**Gegen die Fehlanwendungen**

Die drei ergänzen sich genau. Das erste weckt die Begeisterung, das zweite liefert das Handwerk, und das dritte schützt vor dem, was aus der Begeisterung ohne das Handwerk wird.

Zwei Ergänzungen: [16] Nagel und Newman, wenn Sie den Gödelbeweis in hundert Seiten verstehen wollen. Und [26] Thompson, drei Seiten, die man in zwanzig Minuten liest und danach nie vergisst.